

实验概述

1. 实验目的

- (1) 熟悉SoC的架构及工作原理；
- (2) 理解系统总线的工作原理，掌握总线控制器的设计与实现方法；
- (3) 熟悉I/O接口电路的结构及工作原理，掌握其设计与实现方法。

2. 实验内容

本实验要求在FPGA开发板（芯片型号、引脚约束等详见[开发板使用须知](#)）上，基于单周期或流水线CPU，设计实现支持运行CoreMark或LLAMA2推理程序的SoC（System on Chip，片上系统）。

具体要求如下：

- (1) 为CPU实现ICache和DCache；
- (2) 使用状态机设计实现支持AXI协议的总线控制器；
- (3) 为SoC添加总线桥及主存模块；
- (4) 完成 I/O接口测试程序 的编写，并在SoC上完成这些程序的 下板测试；
- (5) 实现外设I/O接口电路，至少支持5个外设：拨码开关、LED、数码管、UART、计时器；
- (6) 通过调试，使单周期CPU及SoC通过AXI Trace测试；
- (7) 与组员讨论协作，把流水线CPU集成到SoC，并使用AXI Trace调试和验证流水线SoC；
- (8) 与组员讨论协作，在SoC上完成CoreMark或LLAMA2推理程序的下板测试；
- (9) 尝试优化性能，减少CoreMark的运行时间，或提升LLAMA2的推理速率；
- (10) 模板工程默认给SoC提供50MHz的时钟，可根据实际情况自行调节频率（调节方法见[实验1-实验原理-功能部件设计-1.1节的图3-3](#)），要求单周期SoC频率 **不低于25MHz**、流水线SoC频率 **不低于50MHz**。
- (11) 下板检查时，要求不能有时序违例，如图0-1所示。

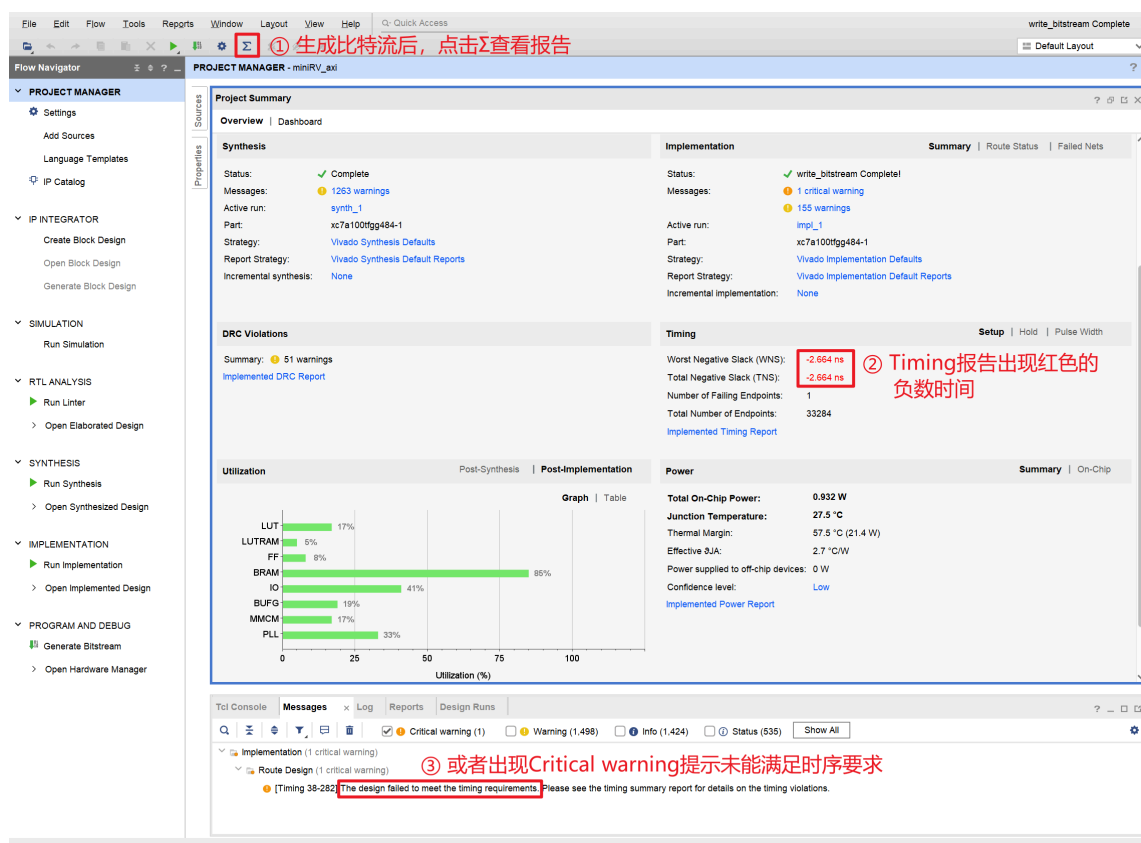


图0-1 生成比特流后，出现时序违例的示意图

实验原理

1. SoC架构

CPU由数据通路和控制器组成，其主要功能是控制和运算。对CPU进行扩展，增加系统总线、存储器和各种各样的I/O接口电路，形成完整的计算机系统，并将整个系统集成到单芯片上，即为SoC。SoC具有集成度高、功耗低、体积小等优点，在嵌入式系统领域中被广泛应用。

本实验提供的SoC架构如图1-1所示。miniRV或miniLA的处理器核心 `cpu_core` 通过其取指接口、数据访问接口与哈佛结构的Cache连接。Cache通过其自身的访存接口与系统总线控制器模块 `axi_master` 连接。

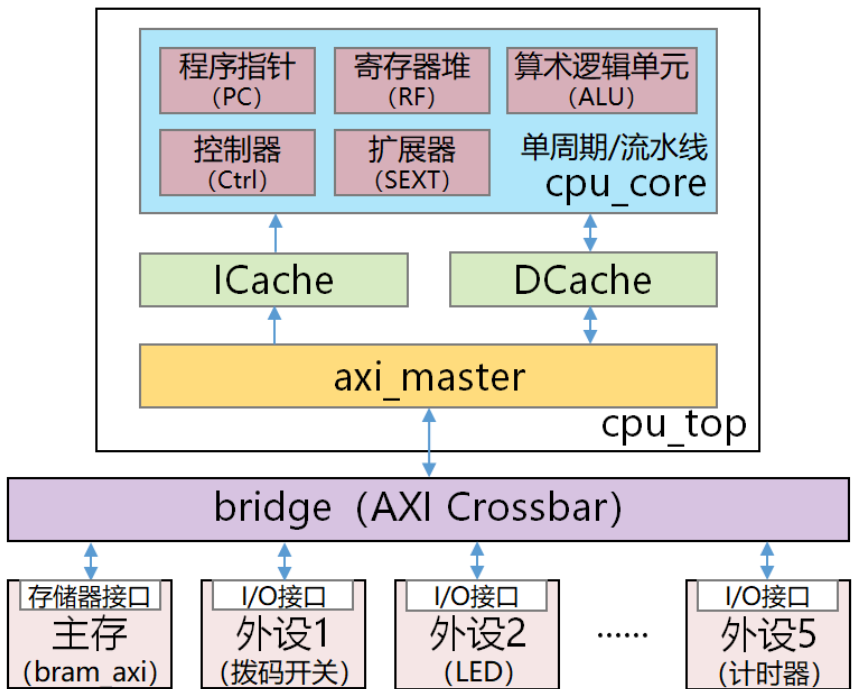


图1-1 miniRV或miniLA SoC架构图

总线控制器 `axi_master` 接收并解析来自CPU或Cache的访问请求，并将其转换成通用的AXI协议的总线读写请求，然后再将这些请求发送到下游的总线桥模块 `bridge`。总线桥根据请求地址，将总线读写请求发送给相应的从设备（主存或外设I/O接口）。通用的总线协议能够快速实现CPU与存储器、外设的互连通信，节约设计和开发成本。

2. `ready - valid` 握手协议

为确保地址、数据等信息能在发送端、接收端之间准确无误地传输，需要在传输信息时添加握手信号。握手信号之间的约定即为握手协议。

ready - valid 协议是最常用的握手协议之一。ready 信号表示接收端的就绪状态 —— 当 ready 信号有效，表示接收端已就绪，可以接收新的信息；否则表示接收端在忙，暂时不能接收信息。valid 信号表示发送端的发送状态 —— 当 valid 信号有效，表示发送端正在发送新的信息，此时信号线上的信息为有效信息；否则表示无信息在发送，此时信号线上的信息是无效信息。

在 ready - valid 握手协议中，当且仅当 ready 信号和 valid 信号均为高电平时，才能在时钟信号上升沿传输数据。ready - valid 握手协议的典型时序如图1-2所示。

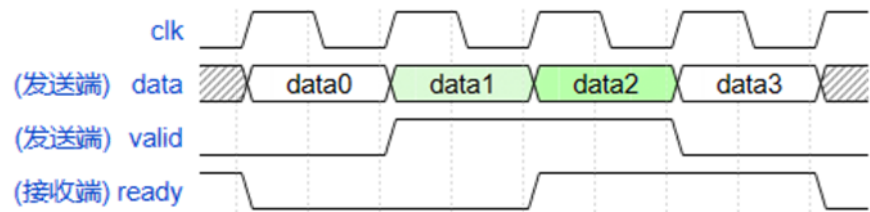


图1-2 ready - valid 握手协议时序图

时序解读

在图1-2中，data0、data3 为无效数据，data1、data2 为有效数据，但只有 data2 被成功发送到接收端。

3. AXI4总线协议

AXI4是ARM提出的基于 ready - valid 握手协议的高性能嵌入式总线协议。

AXI4总线包含写地址（AW）、写数据（W）、写响应（B）、读地址（AR）和读数据（R）五个通道，各通道相互独立且可并行工作，AXI4也因此具备同时传输写请求和读请求的能力。

AXI4总线的读通道相关信号定义如表1-1所示，写通道相关信号定义如表1-2所示。

读通道（AR、R通道）

表1-1 32位AXI4总线的AR、R通道及信号定义

通道	功能	信号	位宽	方向	释义
读地址 (AR)	主设备通过AR通道发送被读数据的地址	arid	4	主→从	发出读请求的主设备ID
		araddr	32	主→从	读地址
		arlen	8	主→从	猝发传输的数据包个数

		arsize	3	主→从	每次传输的数据包大小
		arburst	2	主→从	猝发传输的地址生成方式
		arvalid	1	主→从	读地址的有效信号
		arready	1	从→主	从设备AR通道的就绪信号
读数据 (R)	从设备通过R通道返回被读取数据	rid	4	从→主	当前读响应对应的主设备ID
		rready	1	主→从	主设备R通道的就绪信号
		rdata	32	从→主	读数据
		rresp	2	从→主	读请求响应（可忽略）
		rlast	1	从→主	最后一个读数据的标志位
		rvalid	1	从→主	读数据的有效信号

写通道（AW、W、B通道）

表1-2 32位AXI4总线的AW、W、B通道及信号定义

通道	功能	信号	位宽	方向	释义
写地址 (AW)	主设备通过AW通道发	awid	4	主→从	发出写请求的主设备ID

	送被写数据的地址	awaddr	32	主→从	写地址
		awlen	8	主→从	猝发传输的数据包个数
		awsiz	3	主→从	每次传输的数据包大小
		awburst	2	主→从	猝发传输的地址生成方式
		awvalid	1	主→从	写地址的有效信号
		awready	1	从→主	从设备AW通道的就绪信号
写数据(W)	主设备通过W通道传输待写入到主存或外设的数据	wid	4	主→从	发出写请求的主设备ID
		wdata	32	主→从	写数据
		wstrb	4	主→从	按字节的写使能信号
		wlast	1	主→从	最后一个写数据的标志位
		wvalid	1	主→从	写数据的有效信号
		wready	1	从→主	从设备W通道的就绪信号
写响应(B)	从设备通过B通道返回写请求处理完成的响应	bid	4	从→主	当前写响应对应的主设备ID
		bready	1	主→从	主设备B通道的就绪信

				号
	bresp	2	从→主	写请求响应 (可忽略)
	bvalid	1	从→主	写响应的有效信号

AXI总线支持多个主设备访问多个从设备。为了区分不同的主设备（比如CPU、DMA控制器等），开发者需要 为每个主设备设定一个唯一的ID值。主设备在发出读/写请求时，需将其ID也一并发出。当从设备返回读/写响应时，主设备通过从设备返回的ID判断自己之前发出的读/写请求是否已被从设备处理完毕。

3.1 猝发传输

AXI4总线依靠AR通道的 `arlen`、`arsize` 和 `arburst` 信号，以及R通道的 `rlast` 信号来实现读请求的猝发传输。

`arlen` 信号用于 **设定猝发传输长度**，即一次猝发传输需要连续传输几个数据包（数据包大小由 `arsize` 设定）。若 `arlen` 的值为 i ，表示需要传输 $i + 1$ 个数据包。在AXI4总线中，`arlen` 位宽为8bit，取值范围为 `8'd0 ~ 8'd255`，即猝发传输最多一次连续传输256个数据包。

`arsize` 信号用于设定单个 **数据包的字节数**。若 `arsize` 取值为 i ，则数据包大小为 2^i 字节。在AXI4总线中，`arsize` 位宽为3bit，取值范围为 `3'd0 ~ 3'd7`，即数据包最小为1字节，最大为128字节。

`arburst` 信号用于设定猝发传输时的 **地址生成方式**，其信号取值及作用如表1-3所示。

表1-3 `arburst` 信号取值及其含义

取值	地址生成方式	释义
<code>2'b00</code>	FIXED	固定地址模式，即每次传输的地址相同，适用于重复访问同一地址（如FIFO）
<code>2'b01</code>	INCR	递增地址模式，即每次传输的地址递增，适用于连续访问存储器的场景
<code>2'b10</code>	WRAP	回环地址模式，每次传输地址递增，但达到边界后回到起始地址，边界与 <code>arlen</code> 和 <code>arsize</code> 相关。适用于访问缓存器的场景
<code>2'b11</code>	reserved	保留

主设备在AR通道向从设备发出读地址请求时，通过以上3个信号设置猝发传输的参数。从设备在完成读数据操作后，将按照主设备设定的参数从R通道连续返回若干个数据包，并使用 **rlast** 信号指示**最后一个传输的数据包**。主设备在接收数据时，若检测到 **rlast** 信号有效，则应当在当前数据包接收完毕之后停止接收操作，此时猝发传输结束。具体传输过程见下一节的工作时序。

类似地，AXI4总线的写通道也可以通过设定AW通道的 **awlen**、**awsize** 和 **awburst**，以及W通道的 **wlast** 来支持不同参数的猝发传输，相关原理与读通道类似，此处不再赘述。主要区别是，对于主设备而言，R通道的 **rlast** 是输入信号，而W通道的 **wlast** 是输出信号。

3.2 工作时序

AXI4总线处理读请求、写请求的工作时序分别如图1-3、图1-4所示。

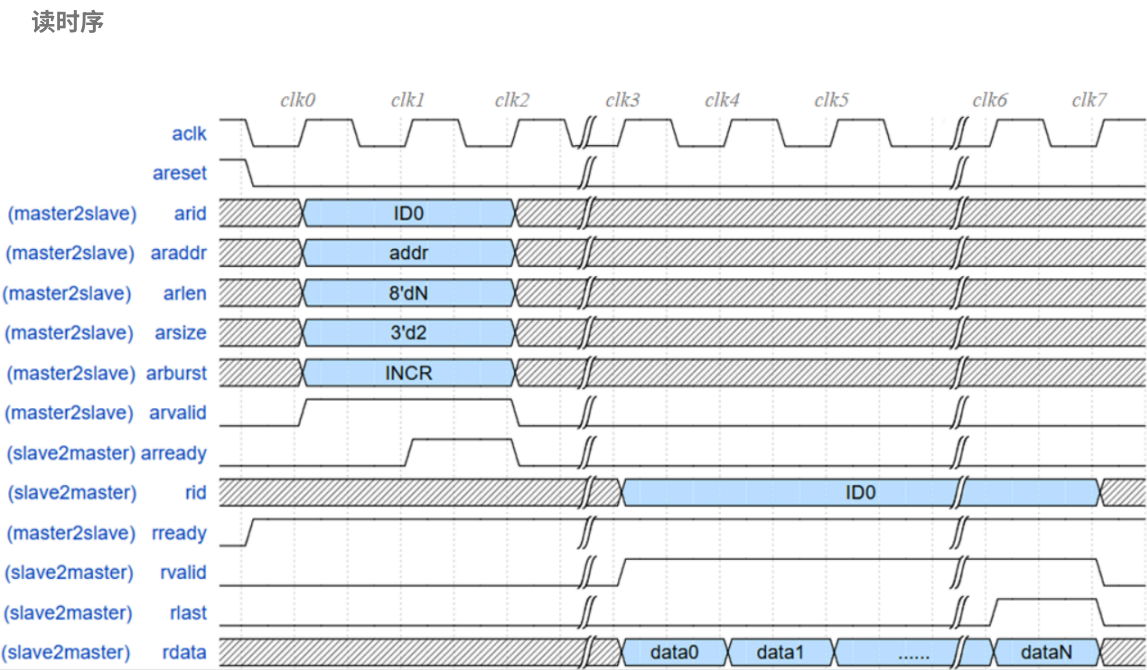


图1-3 AXI4总线读时序

时序解读

- 【clk0】主设备只要有读请求需要发送，就可以把读地址请求发送到AR通道。完整的读地址请求包括 `arid`、`araddr`、`arlen`、`arsize`、`arburst` 和 `arvalid` 6个信号。
- 【clk1】`arready` 信号有效表示从设备就绪，可以接收和处理读请求。因此，**主设备必须一直保持读请求有效，直到从设备的 `arready` 信号有效。**
- 【clk2】主设备在 `clk1` 检测到 `arready` 有效后，在下一个时钟拉低 `arvalid` 信号。读请求成功被从设备接收。
- 【clk3 ~ clk5】从设备完成读操作后，拉高 `rvalid` 信号，同时每个时钟输出一个数据包到 `rdata` 信号。
- 【clk6】从设备输出最后一个数据包时，拉高 `rlast` 标志位信号，表示当前是最后一个数据包，传输即将结束。
- 【clk7】所有数据均传输完毕，从设备拉低 `rvalid` 和 `rlast` 信号。当前读请求处理完成。

写时序

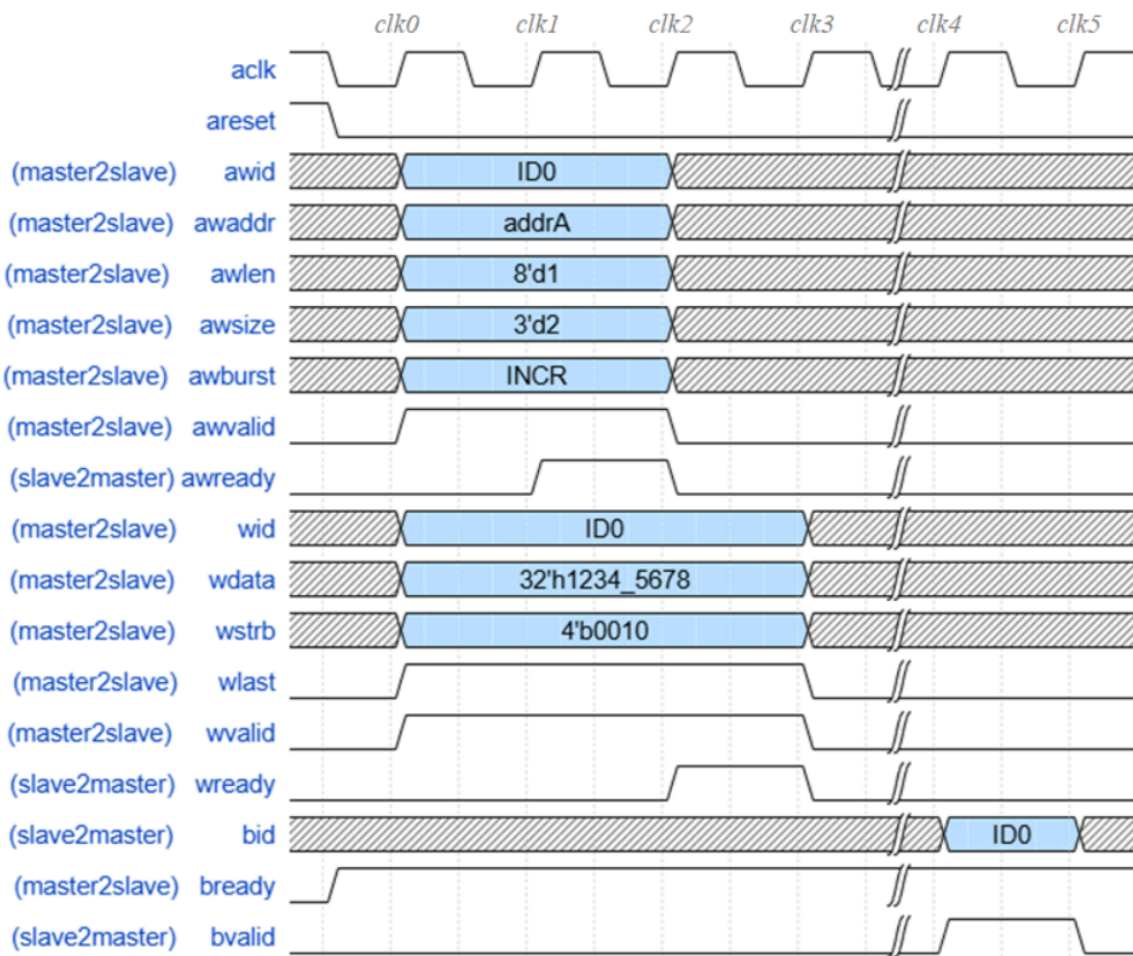


图1-4 AXI4总线写时序

 时序解读

- **【clk0】** 主设备只要有写请求需要发送，就可以把写地址AW请求和写数据W请求分别发送到AW通道和W通道。
完整的AW请求包括 `awid`、`awaddr`、`awlen`、`awsize`、`awburst` 和 `awvalid` 6个信号。
完整的W请求包括 `wid`、`wdata`、`wstrb`、`wlast` 和 `wvalid` 5个信号。
- **【clk1】** `awready` 信号有效表示从设备的AW通道就绪，可以接收和处理写地址请求。因此，**主设备必须一直保持**
AW请求有效，直到从设备的 `awready` 信号有效。
- **【clk2】** 从设备接收主设备的AW请求后，拉低 `awready`，此时主设备应拉低 `awvalid`，以免重复发送AW请求。
此外，`wready` 信号有效表示从设备的W通道就绪，可以接收和处理写数据请求。因此，**主设备必须一直**
保持W请求有效，直到从设备的 `wready` 信号有效。
- **【clk3】** 从设备接收主设备的W请求后，拉低 `wready`，此时主设备应拉低 `wvalid`，以免重复发送W请求。
- **【clk4】** 写操作处理完成后，从设备拉高 `bvalid` 信号。
- **【clk5】** 写响应信号仅有效1个时钟。从设备拉低 `bvalid`，当前写请求处理完成。

主设备一般可同时发出AW请求和W请求，但从设备接收请求的行为可能是不确定的。从设备有时可能同时接收AW请求和W请求，有时可能先接收其中一个；并且如果先接收其中一个请求，则间隔多少个时钟后再接收另一个请求也是不确定的。但 **只有当AW和W通道的请求均被从设备接收后，主设备发出的写请求才被从设备成功接收并处理。**

图1-4所示的波形仅描述了主设备每次写1个存储字的情形，该波形的功能足够满足Load、Store指令的需求。但如果需要实现写数据的猝发传输（比如实现DMA），则应在 `wready` 信号有效时才能传输写数据：实际应用场景中，`wready` 信号不一定一直维持有效，但 `wready` 每有效一个时钟，主设备就传输1个写数据。主设备在传输数据期间，需一直维持 `wvalid` 信号有效，且在传输最后一个数据时，令 `wlast` 信号有效。

为简化设计，可令 `rready`、`bready` 在复位后一直有效，并忽略 `rresp` 和 `bresp`。此外，可用常量驱动 `arid`、`awid` 和 `wid` 信号。

4. Cache的访存接口

在图1-1的SoC架构图中，Cache通过其访存接口与总线控制器 `axi_master` 连接。Cache的访存接口可分为读访存、写访存两组接口信号，相关信号定义如表1-4所示。

表1-4 Cache访存接口的信号定义

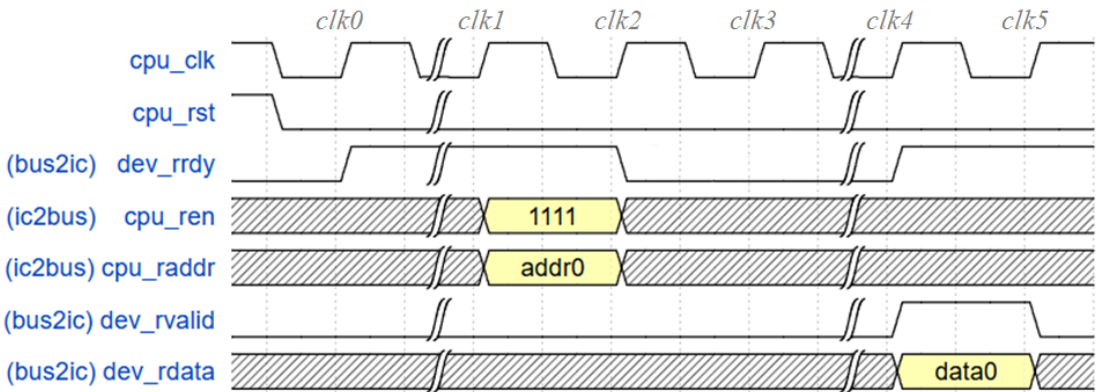
接口	信号	位宽 (bit)	方向	释义

读访存	dev_rrdy	1	总线→Cache	总线读通道的就绪信号
	cpu_ren	1	Cache→总线	读使能
	cpu_raddr	32	Cache→总线	读地址
	dev_rvalid	1	总线→Cache	读数据的有效信号
	dev_rdata	128	总线→Cache	读数据
写访存	dev_wrdy	1	总线→Cache	总线写通道的就绪信号
	cpu_wen	4	Cache→总线	按字节的写使能
	cpu_waddr	32	Cache→总线	写地址
	dev_wdata	32	Cache→总线	写数据

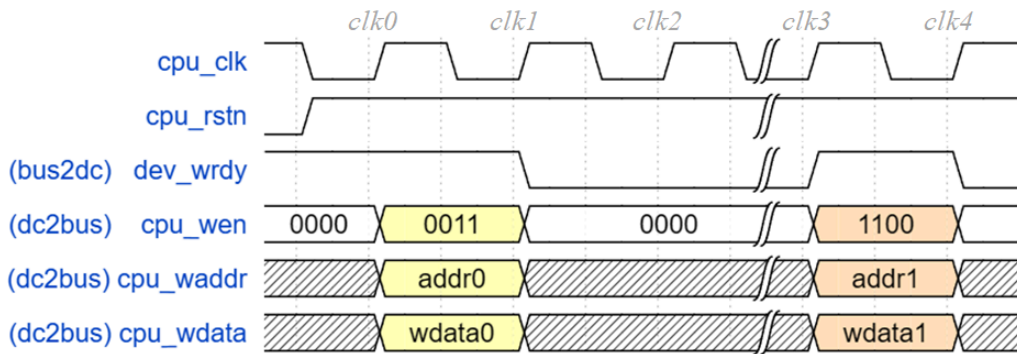
注：总线控制器每次返回一个数据块给Cache。此处假设数据块大小是128位，故 dev_rdata 位宽是128bit。

Cache访存接口的读时序和写时序已在计算机组成原理的实验3中介绍过，此处不再赘述。

计算机组成原理实验3-实验原理-3.2 ICache接口信号、4.1 DCache接口信号



“图2-7 ICACHE面向总线的接口信号时序”



“图2-13 DCache面向总线的写接口时序”

Cache访存总线与 ready - valid 协议

事实上，Cache访存总线协议可看作 ready-valid 协议的简化版变体。在Cache访存总线中，主设备的使能信号 `cpu_ren` 和 `cpu_wen` 兼有 valid 信号的作用 —— 使能信号为 4'b0 时不需要进行读/写操作，相当于表示此时的读/写地址无效。

总线控制器 `axi_master` 的就绪信号 `dev_rrdy` 有效时，Cache的读请求（读使能 `cpu_ren`、读地址 `cpu_raddr`）才会被接收。`axi_master` 接收读请求后，拉低 `dev_rrdy` 并对下游设备进行读操作。数据被取回后，`axi_master` 在拉高 `dev_rvalid` 信号的同时，把数据块通过 `dev_rdata` 信号返回给Cache。`dev_rvalid` 和 `dev_rdata` 只有效1个周期。当读请求处理完成后，`dev_rrdy` 信号被再次拉高。

类似地，`axi_master` 的就绪信号 `dev_wrdy` 有效时，Cache的写请求（写使能 `cpu_wen`、写地址 `cpu_waddr`、写数据 `cpu_wdata`）才会被接收。`axi_master` 接收写请求后，拉低 `dev_wrdy` 并对下游设备进行写操作。写操作完成后，`axi_master` 重新拉高 `dev_wrdy` 信号，等待处理下一个写请求。

`axi_master` 同时包含ICache的读访存接口，以及DCache的读/写访存接口，见下列代码。带有 `ic_`、`dc_` 前缀的信号分别是ICache、DCache的接口信号。

```
axi_master.v
```



```
// ICache Interface
output reg      ic_dev_rrdy  ,
input  wire     ic_cpu_ren   ,
input  wire [ 31:0] ic_cpu_raddr ,
output reg      ic_dev_rvalid,
output reg [ `CACHE_BLK_SIZE-1:0] ic_dev_rdata,
// DCache Write Data Interface
output reg      dc_dev_wrdy  ,
input  wire [ 3:0] dc_cpu_wen  ,
input  wire [31:0] dc_cpu_waddr ,
input  wire [31:0] dc_cpu_wdata ,
// DCache Read Data Interface
output reg      dc_dev_rrdy  ,
input  wire     dc_cpu_ren   ,
input  wire [31:0] dc_cpu_raddr ,
output reg      dc_dev_rvalid,
output reg [ `CACHE_BLK_SIZE-1:0] dc_dev_rdata,
```

一般地，在总线控制器中，**数据写请求优先于数据读请求，而数据读请求又优先于取指请求**。因此，`axi_master` 若同时检测到ICache和DCache的访存请求，需先缓存ICache访存请求的相关信号，等待DCache访存请求处理完毕后，再处理ICache的访存请求。如果只希望实现基础的总线功能，可在检测到多个请求同时存在时，只处理优先级最高的请求，从而简化设计。当然，这样的总线没有任何并行处理能力，其访存性能较低。

I/O接口与外设

1. 接口概述

1.1 接口概念

接口是CPU与“外部世界”的 **连接电路**，负责在CPU与“外部世界”之间“**中转**”各种信息。

在计算机系统中，接口 **介于系统总线与外部设备之间**，可分为 **存储器接口** 和 **I/O接口** 两类——存储器接口是CPU与存储器（如主存）之间的连接电路；I/O接口则是CPU与外部设备之间的连接电路，如图2-1所示。

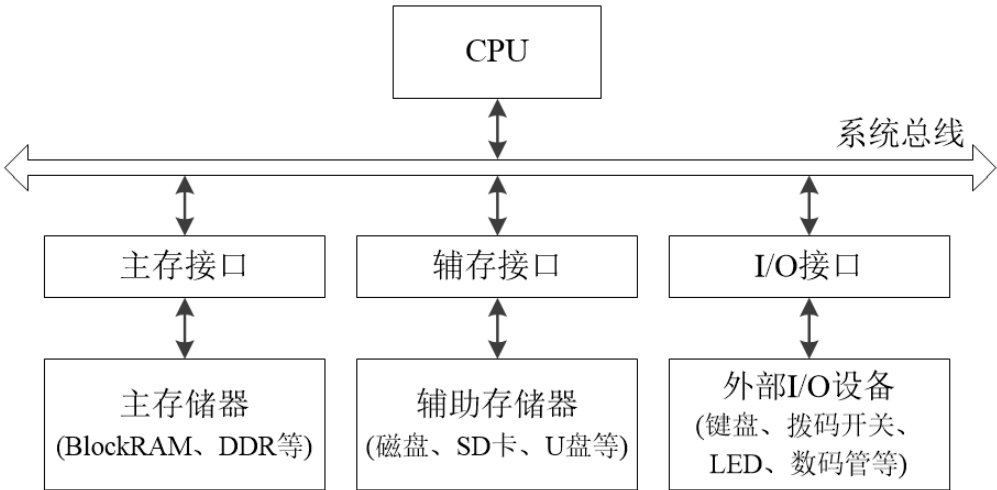


图2-1 计算机系统中的接口

1.2 接口的作用与功能

当CPU需要从“外部世界”获取信息，或向其输出信息时，就需要依靠各式各样的外部设备。

现今使用最广泛的计算机是数字计算机。在数字计算机中，指令和数据都以数字信号“0”和“1”进行表示、传输、运算和存储。而外部设备不仅种类多、功能杂，其数据格式、工作原理也各不相同。为了使得CPU能够高效、规范地使用外部设备，我们需要为外部设备设计相应的接口，以适配外部设备和CPU之间的差异。

在计算机系统中，接口受CPU控制，外部设备则受接口控制，如图2-2所示。

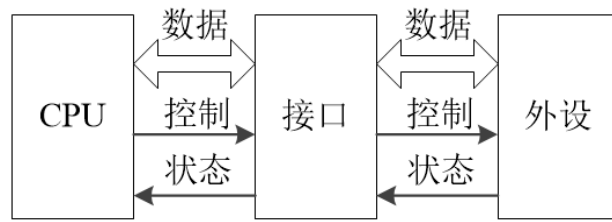


图2-2 接口的作用

补充说明

需要注意的是，对某些外设而言，图2-2中的数据信号线可能是单向的。

接口通常具有以下功能：

- **信号转换**功能（预处理功能）

完成总线信号与I/O设备信号之间的转换，包括信号的逻辑关系、时序配合、电平转换、串并转换等。

- **数据缓存**功能

I/O接口中含有寄存器、锁存器或其他缓存存储器，用于实现CPU和外设之间的数据缓存功能。接口中用于暂存数据的缓存器称为I/O接口对CPU的 **数据口**。

- **接受和执行CPU命令**功能

I/O接口中含有存放CPU命令码的寄存器，该寄存器称为I/O接口对CPU的 **命令口**。接口工作时，将根据命令口中的命令码，产生相应的控制信号，控制外设完成不同的操作。

- **控制和监视外设执行**功能

I/O接口中还含有存放外设状态信息的寄存器，该寄存器称为I/O接口对CPU的 **状态口**。接口工作时，将实时监视外设的执行状态，并适时更新状态寄存器。CPU通过读取状态口，即可获取外设的执行状态。有时I/O接口也需要根据状态寄存器的值来产生外设工作所需的控制信号。

- **设备选择**功能（选址功能）

CPU通过不同的地址访问不同的I/O接口以及接口中的设备。I/O接口通过地址译码器，产生设备选择信号，以选择对应的外设与CPU进行通信。

补充说明

- (1) 接口和设备不一定是一对一的关系。一个接口可能含有多个数据口、命令口和状态口。
- (2) 每一个数据口、命令口或状态口均可由CPU通过地址直接访问。

1.3 I/O接口电路

I/O接口电路的核心是数据口、命令口、状态口和控制逻辑，其基本结构如图2-3所示。

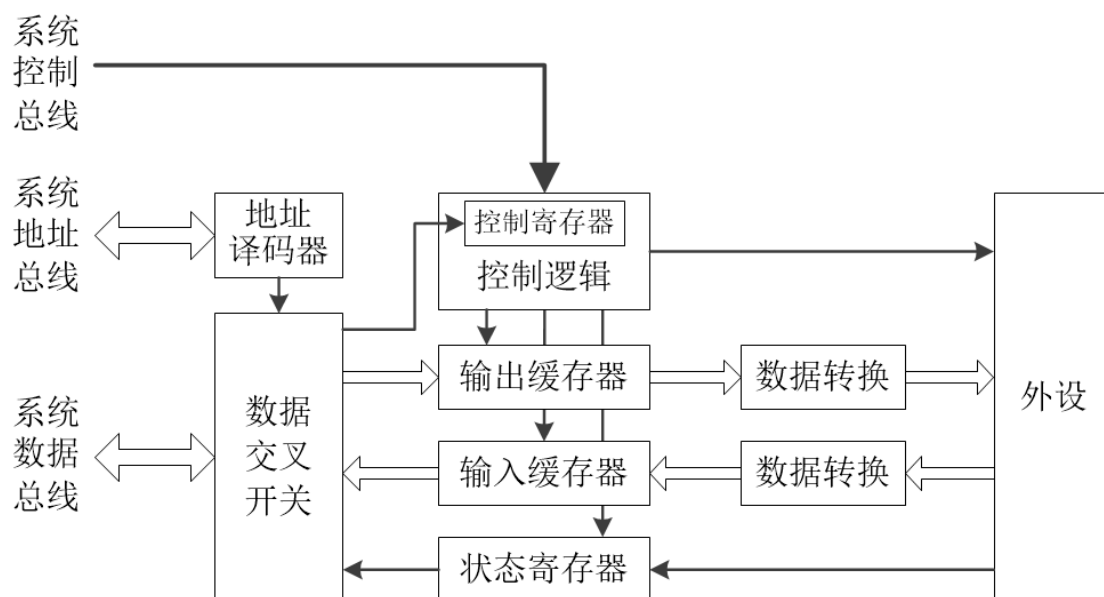


图2-3 I/O接口电路的结构

由图2-3可知，CPU通过系统总线实现与I/O接口电路的交互。显然，CPU需要通过地址来区分当前访问的是命令口、数据口还是状态口。

举个栗子 🌰

假设某外设接口的基地址为 `0xFFFF_FFA0`，且命令口、数据口和状态口的偏移地址分别为 `2'b00`、`2'b01` 和 `2'b10`。

当CPU想初始化该外设时（需要访问命令口），需要向 `0xFFFF_FFA0` 写入初始化命令。此时，地址 `0xFFFF_FFA0` 将通过系统地址总线传输到接口电路的地址译码器；初始化命令所对应的二进制编码数据将通过系统数据总线传输到接口电路的数据交叉开关；“写”信号将通过系统控制总线传输到接口电路的控制逻辑。然后，地址译码器对收到的 `0xFFFF_FFA0` 地址进行译码，发现当前地址的高30位等于 `30'h3FFF_FFE8` 且最低2位等于 `2'b00`，说明此时被访问的是控制寄存器，则初始化命令将经过数据交叉开关，写入到控制寄存器。接下来，控制逻辑将根据控制寄存器的值，产生控制信号，控制外设完成初始化操作。

类似地，当CPU想读取该外设的运行状态信息时（需要访问状态口），需要读取 `0xFFFF_FFA2` 地址。此时，地址 `0xFFFF_FFA2` 将通过系统地址总线传输到接口电路的地址译码器；“读”信号将通过系统控制总线传输到接口电路的控制逻辑。然后，地址译码器对 `0xFFFF_FFA2` 的地址进行译码，发现当前地址的高30位等于 `30'h3FFF_FFE8` 且最低2位等于 `2'b10`，说明此时被访问的是状态寄存器。接下来，控制逻辑将产生控制信号，控制外设将最新的状态信息写入状态寄存器，然后状态寄存器的新值将经过数据交叉开关，写入到系统数据总线。

2. 基本外设

2.1 外设的编址方式

I/O设备（即外设）通常有独立编址、统一编址两种编址方式。“独立”和“统一”指的是外设地址和主存地址的关系 —— 在独立编址模式中，外设的地址空间完全独立于主存地址空间之外，CPU需要通过特殊的IO指令才能访问外设；在统一编址模式中，外设和主存共用同一片地址空间，CPU可使用访存指令来访问外设。

miniRV和miniLA的外设均采用统一编址方式。32位的地址空间被划分成主存地址空间和I/O地址空间2部分 —— **最高64KB是I/O设备的地址空间（即0xFFFF_0000 ~ 0xFFFF_FFFF）**，其余则是主存地址空间（即0x0000_0000 ~ 0xFFFE_FFFF）。理论上，程序可用的最大主存空间为4GB减去64KB，实际应考虑主存物理介质的实际容量大小。

2.2 外设基地址及端口地址

外设I/O接口的地址由 **基地址** 和 **偏移地址** 组成。基地址相当于外设的身份ID，决定了CPU访问的是哪个外设。偏移地址则决定CPU访问外设的哪个端口。

因此当程序希望控制外设完成某个特定的功能时，程序需要通过访存指令来访问特定的端口。

在本课程的SoC中，外设I/O接口的基地址和端口地址如表2-1所示。其中，地址信号的高20位决定外设的基地址，低12位决定端口地址。不同的端口通常用于实现不同的功能。例如，计时器I/O接口的基地址是0xFFFF_4000，并且具有0x000、0x008两个端口，这两个端口的功能分别是读取计时器的低32位和高32位。

表2-1 基本外设基地址及端口说明

外设	端口数	端口地址	端口属性	端口功能
拨码开关	1	0xFFFF_0000	只读	提供外部输入
LED	1	0xFFFF_1000	写	显示数据
数码管	1	0xFFFF_2000	写	显示数据
UART	4	0xFFFF_3000	只读	接收字符的队列缓存器，即RX FIFO
		0xFFFF_3004	写	发送字符的队列缓存器，即TX FIFO
		0xFFFF_3008	读	状态寄存器，用于记录RX FIFO和TX

			FIFO的状态等信息
计时器	2	0xFFFF_300C	写 控制寄存器，用于初始化及控制UART
		0xFFFF_4000	只读 读取64位计时器的低32位
		0xFFFF_4008	只读 读取64位计时器的高32位

2.3 I/O接口的读写访问

在RISC-V或LoongArch的RISC指令集架构中，只有Load指令和Store指令可以访存，且主存与外设统一编址。因此，在汇编程序里，访问主存和访问外设的差别仅仅是数据读写的地址不同。

一个外设的I/O接口可能包含多个不同的端口。程序希望控制外设完成某个特定的功能时，需要通过访存指令来读写相应的端口。

在C语言里，通常使用指针来访问外设。例如，由表2-1可知，数码管、计时器的I/O接口基地址分别是0xFFFF_2000、0xFFFF_4000，则有下列示例程序。

```
// 外设I/O接口的基地址
#define PERI_DLED_ADDR 0xFFFF2000 // 数码管
#define PERI_TIMER_ADDR 0xFFFF4000 // 计时器

// I/O接口中，各端口的偏移地址
volatile unsigned int *peri_digled = (volatile unsigned int*) PERI_DLED_ADDR;
volatile unsigned int *peri_timer_lo32 = (volatile unsigned int*)(PERI_TIMER_ADDR + 0x000);
volatile unsigned int *peri_timer_hi32 = (volatile unsigned int*)(PERI_TIMER_ADDR + 0x008);

int main()
{
    // 读取计时器高32位的值
    unsigned int timer_high = *peri_timer_hi32;

    // 把数据显示到数码管
    *peri_digled = timer_high;

    return 0;
}
```

其中，volatile 关键字用于防止编译器优化，保证CPU能够及时获取到外设的数据。每个外设的端口地址都被定义成相应的 volatile 指针，读写数据时，对指针解引用后再进行读写即

可。

2.4 常用外设简介

FPGA开发板通常具有一定的外设资源，常见的外设包括12位VGA视频接口、10/100/1000Mbps以太网、蜂鸣器、麦克风、Micro SD卡、UART串口、矩阵键盘、数码管、LED、拨码开关、按键开关等。

以下介绍Minisys及EGO1常用的几种外设。

2.4.1 拨码开关和LED

拨码开关和LED是最基本的输入输出设备，常用于辅助硬件调试。

拨码开关拨到下档时，表示向对应的FPGA引脚输入低电平，否则输入高电平。LED则是从对应的FPGA引脚接收到高电平时点亮，否则熄灭。

Minisys

Minisys开发板具有24个拨码开关和24个LED，如图2-4所示。

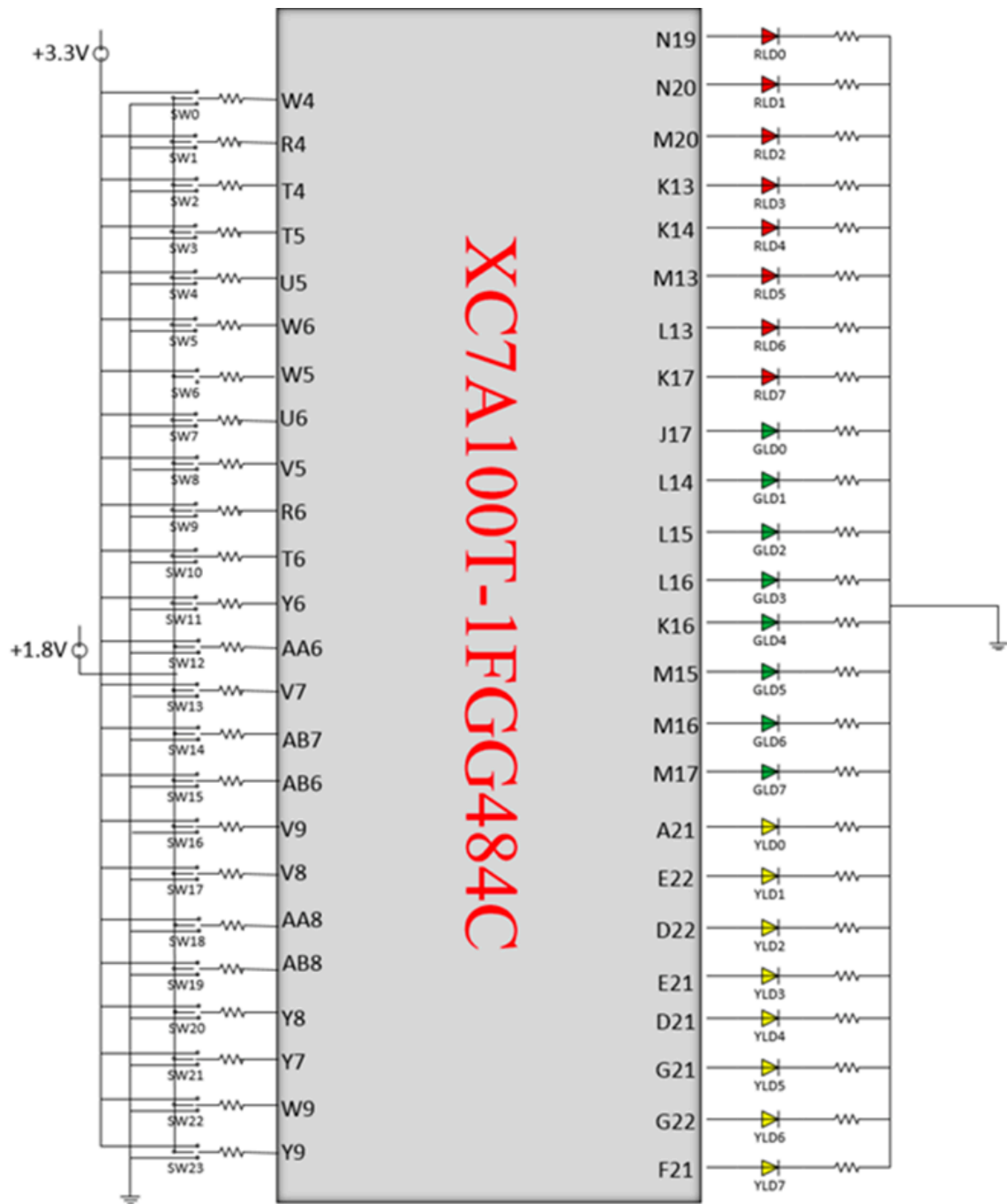


图2-4 Minisys开发板的拨码开关与LED电路图

EGO1

EGO1开发板包含16个拨码开关和16个LED，其中，16个拨码开关分成2组，左边8个是正常尺寸的拨码开关，右边8个是集成在一起的DIP小开关，如图2-4A所示。

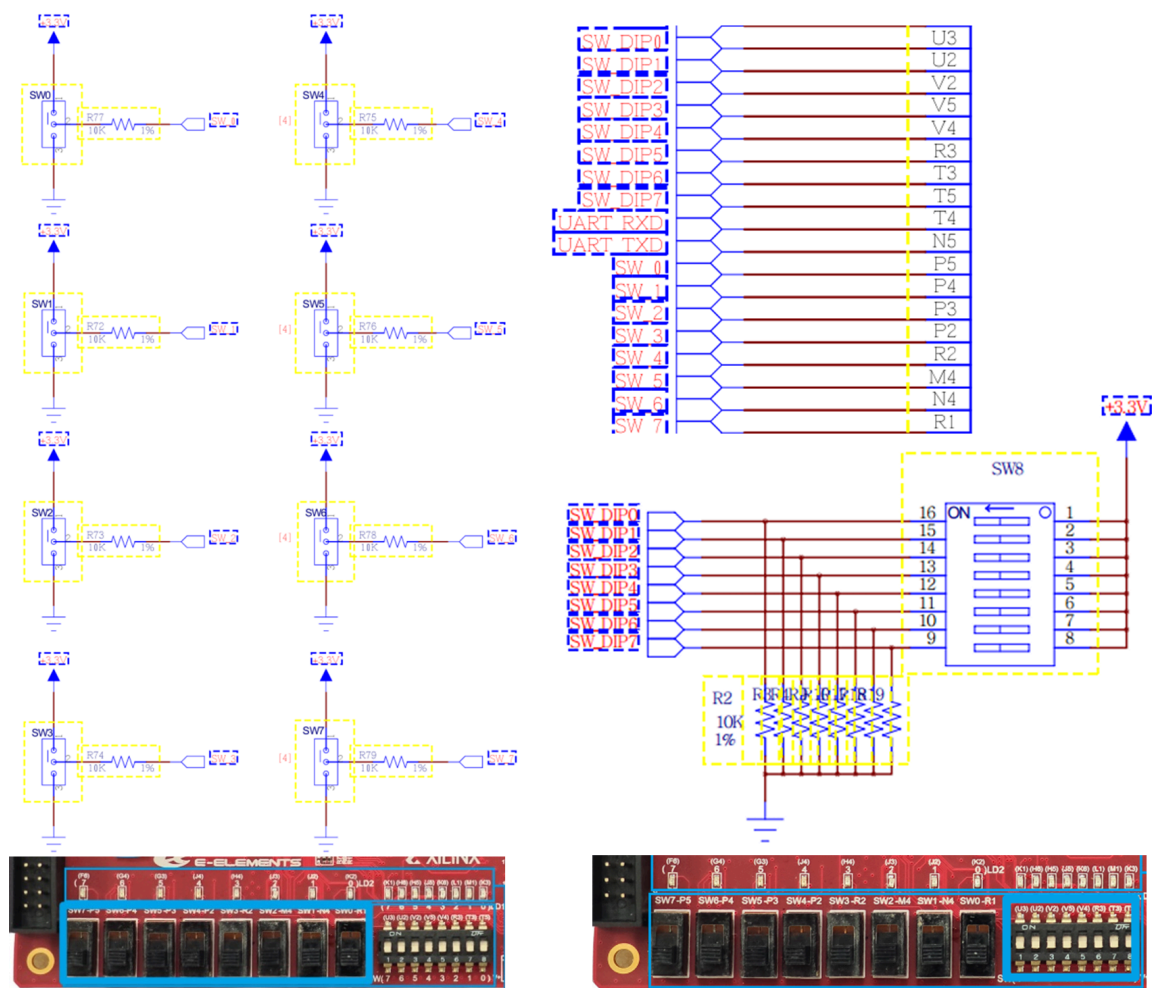


图2-4A EGO1开发板的拨码开关与LED电路图

2.4.2 数码管

Minisys开发板有两个4位带小数点的七段数码管，其电路原理图如图2-5所示。其中，A7-A0是数码管8个位的使能信号（即 位选信号），而CA-CG及DP则对应各个位上的七个段以及小数点的触发信号（即 段选信号）。

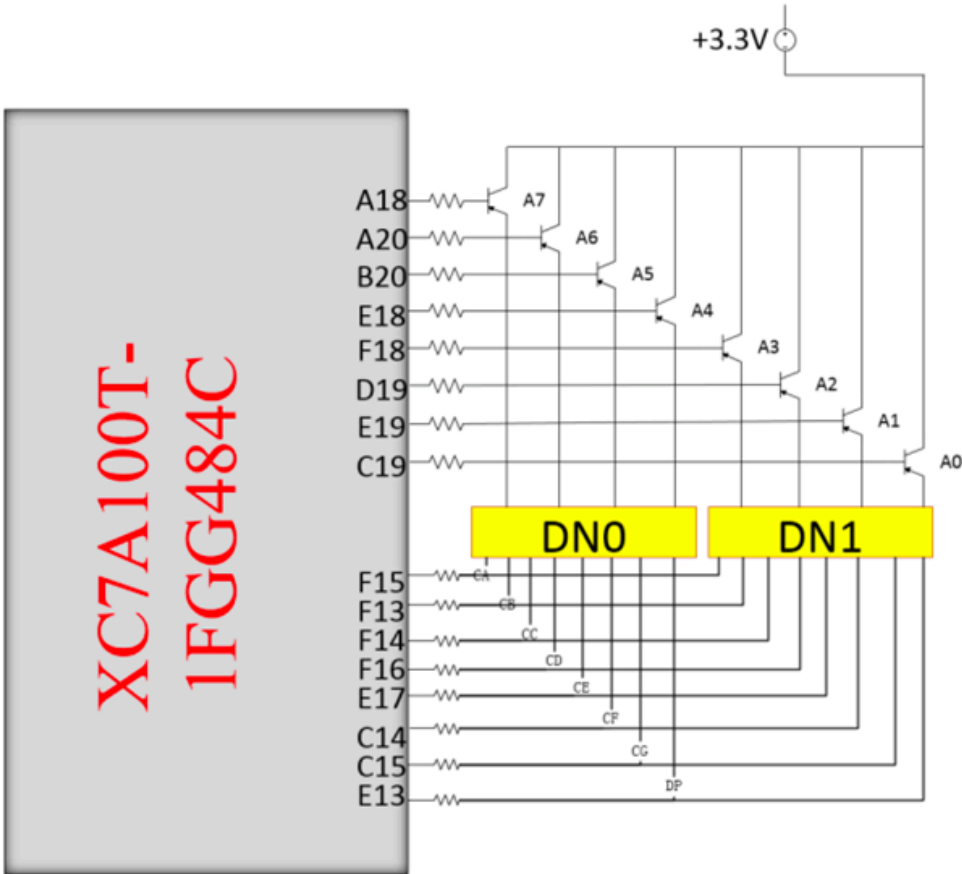


图2-5 七段数码管与FPGA芯片的连接图



注意数码管的有效电平

Minisys开发板 的数码管位选信号和段选信号均为 **低电平有效**。

EGO1开发板 的数码管位选信号和段选信号均为 **高电平有效**。



关于EGO1开发板的数码管

EGO1开发板的8位数码管分为2组 —— 左边4位为第一组，右边4位为第二组。两组数码管具有各自独立的段选信号。实现时，可使用 `assign` 语句，令第二组数码管的段选信号始终等于第一组的段选信号。

数码管本质上是LED灯。图2-6以最后侧的A0数码管为例，说明了7段数码管的内部结构。

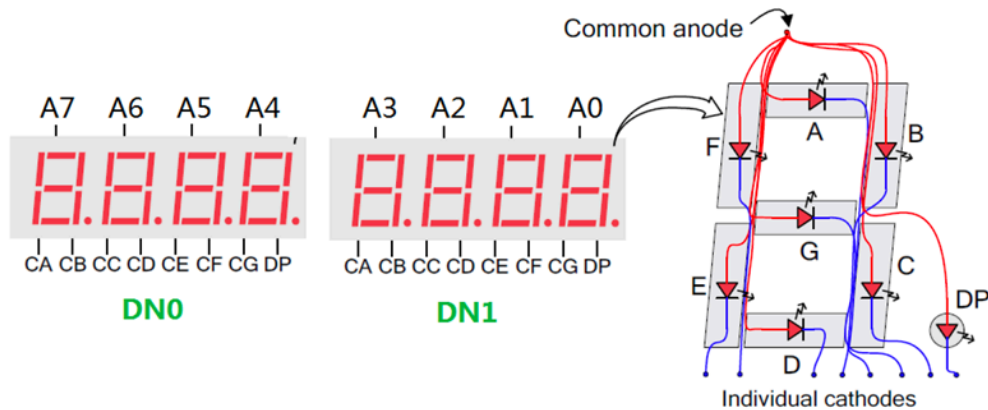


图2-6 七段数码管结构图

在图2-6中，A0数码管的A段连接到DN1的CA引脚，B段连接到DN1的CB引脚，以此类推。CA-CG及DP均为低电平有效，且A7-A4、A2-A0分别共用一组段选信号。因此，如果给DN0的CA引脚提供低电平时，数码管A7-A4的A段将被同时点亮；如果给DN1的CB引脚提供低电平时，数码管A2-A0的B段将被同时点亮。

此外，每一位数码管都有一个使能信号，因此使能信号位宽为8（A[7:0]）。A[7:0]通过反相器接到对应数码管的每一个段的阳极上，因此A[7:0]也是低电平有效的。

2.4.3 UART

UART串口是常用的嵌入式低速通信外设，只需TX、RX两根信号线即可在收发双方之间实现全双工通信。

UART通过数据帧通信。一个数据帧包含起始位、数据位、校验位和停止位，如图2-7所示。

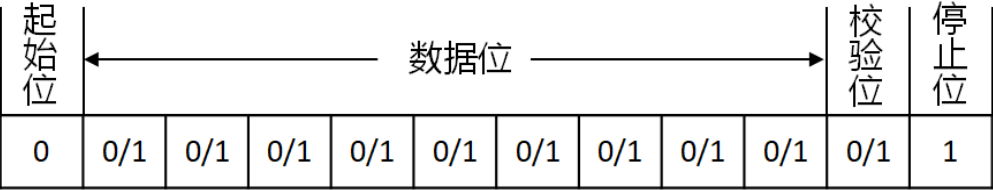


图2-7 UART数据帧格式

数据帧最常用的配置是“8N1”，即8个数据位、无校验位、1个停止位。

数据帧的每个位都需要按照波特率要求，维持一定的电平有效时间。比如波特率如果设置为115200bps，则数据帧的每个位需要维持 $\frac{1}{115200} \text{s} \approx 8.68\mu\text{s}$ 。

2.4.4 计时器

计时器也称定时器、定时计数器，是计算机硬件系统的常用外设。计时器能够实现 *ns* 级别的高精度计时，不仅能够给计算机系统提供时间基准，还能结合CPU的例外与中断处理机制，为操作系统的任务调度、延迟操作等功能提供底层硬件支持。

在本实验中，计时器由一个64位的计数器构成。复位之后，该计时器自动开始计数。

本课程的计时器外设具有2个端口（见表2-1），地址偏移量分别为 0x00 和 0x08 ，分别用于读取计数器的低32位、高32位的值。

2.4.5 按键开关

开发板共有5个可供开发者自定义的按键开关，如图2-8所示。当某一按键按下时，其对应的FPGA输入为1，否则为0。

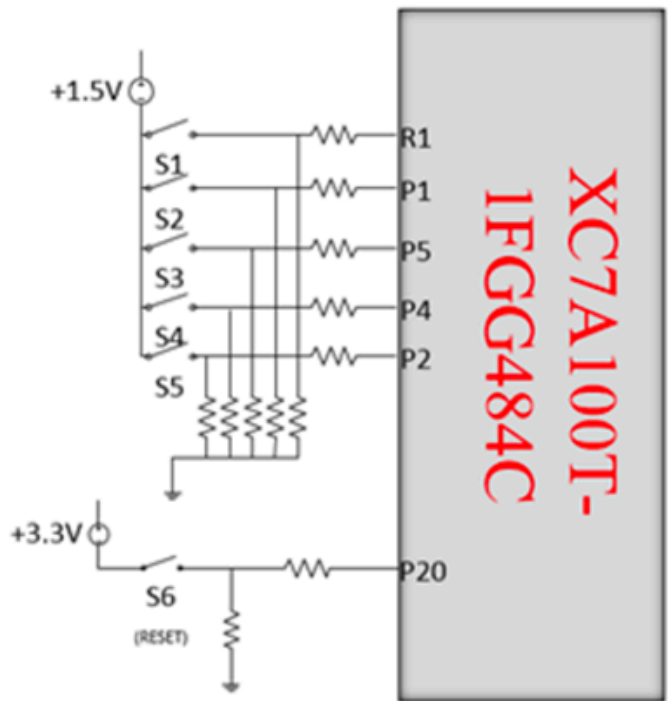


图2-8 Minisys开发板的按键开关连接图

按键开关内部包含金属弹簧，按下和弹起时，金属弹簧发生振动，从而导致电平不稳定，此种现象称为按键开关的抖动。一般可采用计数器延迟的方法进行消抖，其基本思路是，检测到按键开关信号发生变化时，启动计数器延迟10~15ms，并再次读取按键开关的电平，如图2-9所示。

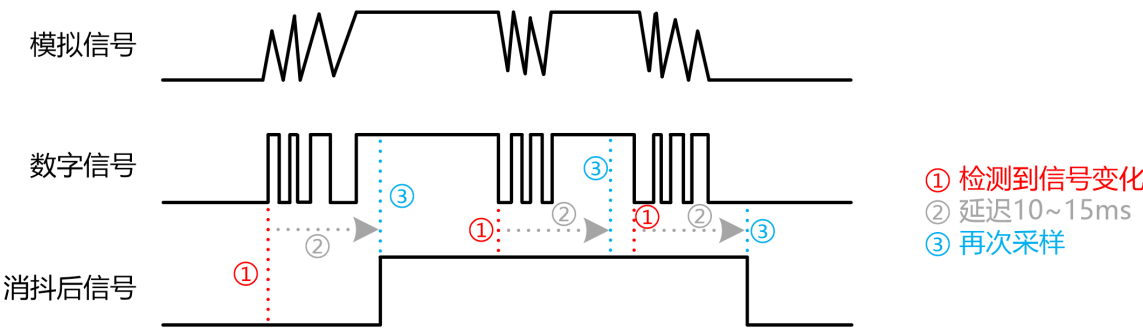


图2-9 按键信号的消抖原理示意图

2.4.6 4×4键盘

Minisys开发板的4×4键盘通过4根行选线和4根列选线连接到FPGA芯片，如图2-10所示。

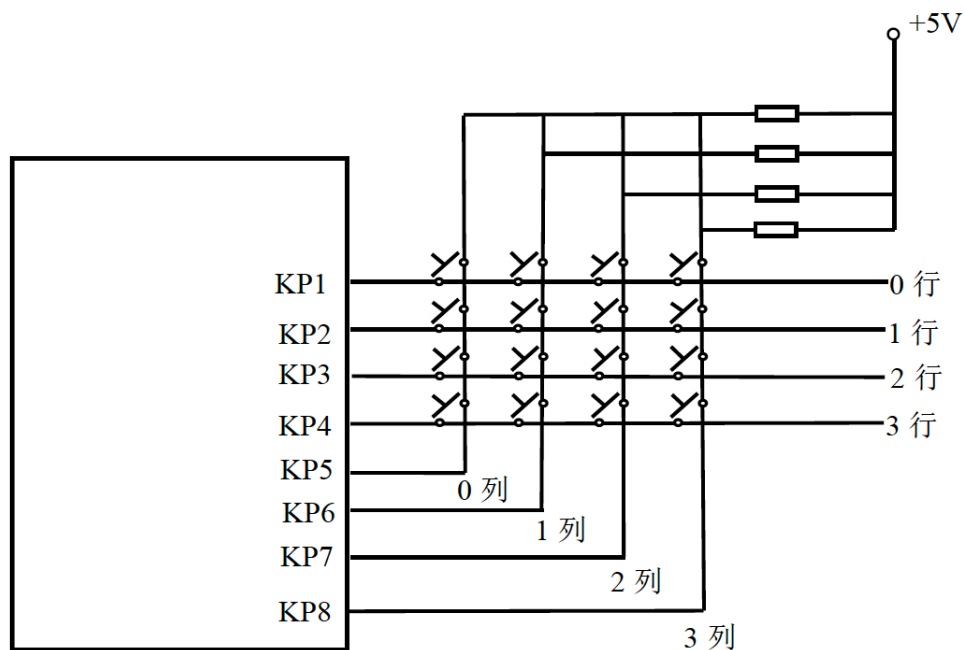


图2-10 矩阵键盘的电路原理图

4×4键盘的基本原理是行列扫描。实现时，可通过移位寄存器或计数器生成4位流水灯信号，输出到行信号，然后读取列信号。通过判断行信号与列信号的取值，即可得出被按下按键的坐标。然后把按键坐标映射到对应的键值，即需要在程序中对扫描得到的坐标行、列信号的各种组合方式进行翻译。

实验步骤

- (1) 把单周期CPU的工程及代码复制到另一个文件夹，然后在其基础上开发总线控制器和I/O接口电路；
- (2) 把计算机组成原理实验3所实现的ICache、DCache模块集成到单周期CPU的工程当中；
- (3) 使用状态机实现AXI总线控制器模块 `axi_master`，添加总线桥和主存IP核；
- (4) 完成I/O接口编程的C_TEST测试程序，并使用提供的SoC比特流进行下板测试；
- (5) 至少实现 拨码开关、LED、数码管、UART 和 计时器 共5个外设的I/O接口电路；
- (6) 使用Trace测试框架，对包含AXI总线控制器和I/O接口的单周期SoC进行功能验证，使其通过AXI Trace测试；



调试建议

首次使用包含AXI总线的SoC运行Trace测试时，建议先通过 `defines.vh` 头文件禁用Cache。等通过AXI Trace测试之后，启用Cache并再次进行调试。



如果需要在Vivado中运行功能仿真

使用 `bin2coe.py` 脚本，可将Trace测试框架中的 `cdp-tests / bin` 文件夹下的.bin文件转换成.coe文件。

参考[实验1-实验原理-功能部件设计-3.2 Block Memory的初始化-图3-8](#)，把.coe文件导入 `bram_axi` 的IP核当中，再运行功能仿真即可。

- (7) 与组员讨论协作，把流水线CPU集成到SoC，并使用AXI Trace调试和验证流水线SoC；
- (8) 与组员讨论协作，在SoC上完成CoreMark或LLAMA2推理程序的下板测试；
- (9) 尝试优化性能（比如提高频率、优化访存、优化运算等），减少CoreMark的运行时间，或提升LLAMA2的推理速率。

CoreMark运行效果示例

SoC下板运行CoreMark的效果如图3-1所示。

```
CoreMark 1.0
2K performance run parameters for coremark.
CoreMark Size      : 666
Total ticks        : 897896501
Total time (secs)  : 11
Iterations/Sec     : 63
Iterations         : 700
Compiler version   : GCC10.2.0
Compiler flags     : -O2 -funroll-loops -fpeel-loops -fgcse-sm -fgcse-las
Memory location    : STATIC
seedcrc           : 0xe9f5
[0]crclist        : 0xe714
[0]crcmatrix      : 0x1fd7
[0]crcstate       : 0x8e3a
[0]crcfinal       : 0x65c5
Correct operation validated. See README.md for run and reporting rules.
CoreMark 1.0 : 62.3679
CoreMark/MHz : 0.7795
FINISH
```

① 出现此提示，表明运行结果正确

② 性能数据

图3-1 CoreMark下板运行效果示意图

若CoreMark下板运行出现问题，可将C_TEST中的测试程序逐一下板测试，并结合其代码进行调试。

注意：提升CPU时钟频率后，需相应修改C程序的频率参数

提高CPU时钟频率的方法见[常见问题 - 1.7 如何提高CPU主频？](#)

模板工程的CPU时钟默认为50MHz。改变频率后，需要相应地修改C程序中的频率参数，否则程序计时不准确。

具体地，我们需要修改CoreMark源文件 `core_portme.c` 第36行的 `MHZ` 参数，使其等于实际的CPU频率：

`c_test / 4_coremark / src / coremark / core_portme.c`

```
36 #define MHZ 50 // CPU Clock Frequency
37 #define CLOCKS_PER_SEC (1000000 * MHZ)
```

总线控制器实现

首先，为现有的单周期CPU工程创建副本，保存好原有代码及工程。后续所有操作均在副本工程上进行，下文称其为SoC工程。

1. 实现Cache

把计算机组成原理实验3所实现的ICache.v、DCache.v源文件拷贝到单周期SoC工程的 `src / rtl` 目录之内。

打开SoC工程，添加ICache、DCache模块到Design Sources，如图4-1所示。注意不要添加成Simulations Sources或Constraints。

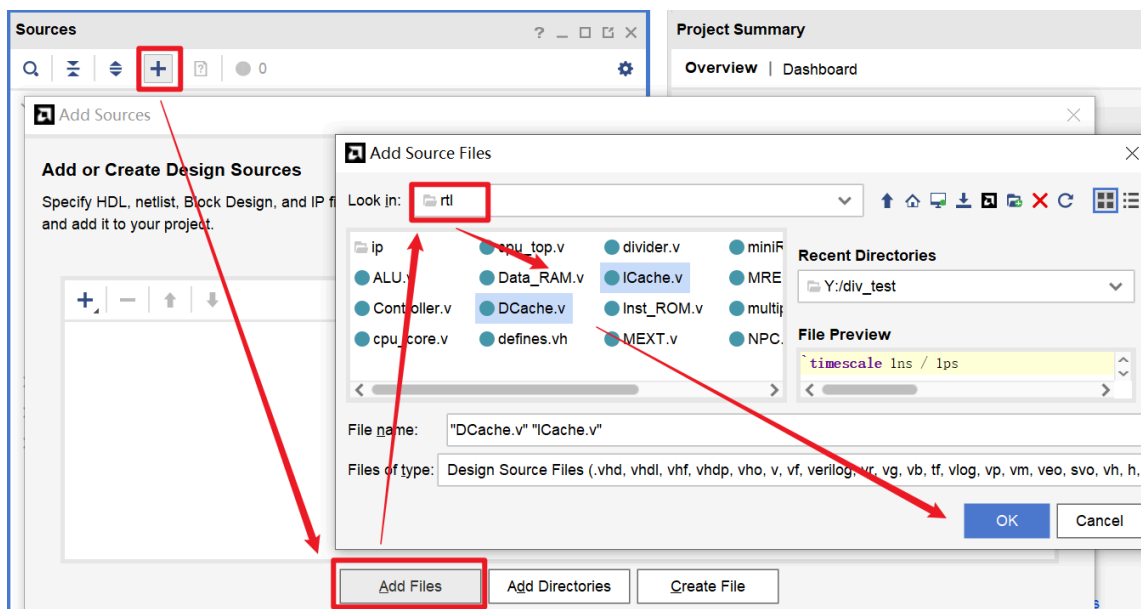


图4-1 添加Cache模块源文件

打开 `cpu_top.v`，删除原有的 `Inst_ROM` 和 `Data_RAM` 模块的实例化代码，并编写ICache、DCache模块的实例化代码。

2. 为SoC添加AXI总线接口

在SoC工程的 `src / rtl` 目录创建 `axi_master.v` 文件，并参照图4-1将其添加到工程当中。

编写总线控制器 `axi_master` 的模块定义：

```
axi_master.v
```



```

`timescale 1ns / 1ps

`include "defines.vh"

module axi_master(
    input wire      aclk,
    input wire      areset,    // high active

    // ICache Interface
    output reg      ic_dev_rrdy,
    input wire      ic_cpu_ren,
    input wire [31:0] ic_cpu_raddr,
    output reg      ic_dev_rvalid,
    output reg [IC_BLK_SIZE-1:0] ic_dev_rdata,
    // DCache Interface
    output reg      dc_dev_wrdy,
    input wire [3:0] dc_cpu_wen,
    input wire [31:0] dc_cpu_waddr,
    input wire [31:0] dc_cpu_wdata,
    output reg      dc_dev_rrdy,
    input wire      dc_cpu_ren

```

在 `cpu_top.v` 中添加 `axi_master` 模块的实例化代码：

cpu_top.v

```

axi_master U_aximaster (
    .aclk      (),
    .areset    (),

    // ICache Interface
    .ic_dev_rrdy  (),
    .ic_cpu_ren   (),
    .ic_cpu_raddr (),
    .ic_dev_rvalid (),
    .ic_dev_rdata (),
    // DCache Interface
    .dc_dev_wrdy  (),
    .dc_cpu_wen   (),
    .dc_cpu_waddr (),
    .dc_cpu_wdata (),
    .dc_dev_rrdy  (),
    .dc_cpu_ren   (),
    .dc_cpu_raddr (),
    .dc_dev_rvalid (),
    .dc_dev_rdata ()

```

为 `cpu_top.v` 添加AXI总线的Master接口：

cpu_top.v

```

module cpu_top(
    input wire      cpu_clk,
    input wire      cpu_rst,    // high active

```

```
// AXI4 Master Interface
// write address channel
output wire [31:0] m_axi_awaddr,
output wire [ 7:0] m_axi_awlen,
output wire [ 2:0] m_axi_awsz,
output wire [ 1:0] m_axi_awburst,
input wire      m_axi_awready,
output wire      m_axi_awvalid,
// write data channel
output wire [31:0] m_axi_wdata,
input wire      m_axi_wready,
output wire [ 3:0] m_axi_wstrb,
output wire      m_axi_wlast,
output wire      m_axi_wvalid,
// write response channel
output wire      m_axi_bready,
input wire [ 1:0] m_axi_bresp,
```

根据信号名，把 `cpu_top` 模块的AXI Master接口信号，连接到 `axi_master` 模块的实例。

打开 `miniRV_SoC.v` 或 `miniLA_SoC.v`，为 `cpu_top` 模块的实例添加AXI接口信号连接：

miniRV_SoC.v 或 miniLA_SoC.v

```
wire [31:0] cpu_awaddr ;
wire [ 7:0] cpu_awlen ;
wire [ 2:0] cpu_awsz ;
wire [ 1:0] cpu_awburst;
wire      cpu_awvalid;
wire      cpu_awready;
wire [31:0] cpu_wdata ;
wire [ 3:0] cpu_wstrb ;
wire      cpu_wlast ;
wire      cpu_wvalid ;
wire      cpu_wready ;
wire      cpu_bready ;
wire [ 1:0] cpu_bresp ;
wire      cpu_bvalid ;
wire [31:0] cpu_araddr ;
wire [ 7:0] cpu_arlen ;
wire [ 2:0] cpu_arsz ;
wire [ 1:0] cpu_arburst;
wire      cpu_arvalid;
wire      cpu_arready;
wire      cpu_rready ;
```

3. 创建AXI接口的主存模块

在SoC工程中，点击界面左侧的 `IP Catalog`，搜索“`block`”。找到 `Block Memory Generator` 后双击打开IP核配置窗口，如图4-2所示。

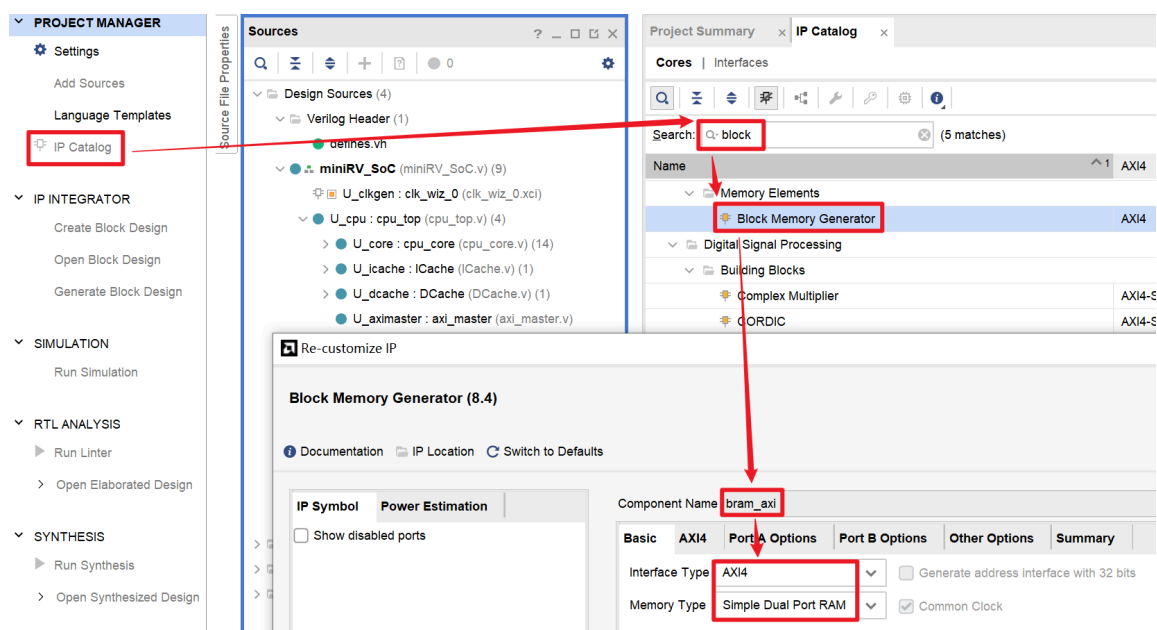


图4-2 创建 bram_axi 的主存IP核

设置IP核的名称为“bram_axi”，设置接口类型为AXI4，并且设置存储类型为 Simple Dual Port RAM。

在图4-2中，点击切换到“Port A Options”标签页，按图4-3所示配置IP核的数据位宽及存储深度（即数据单元的个数）。

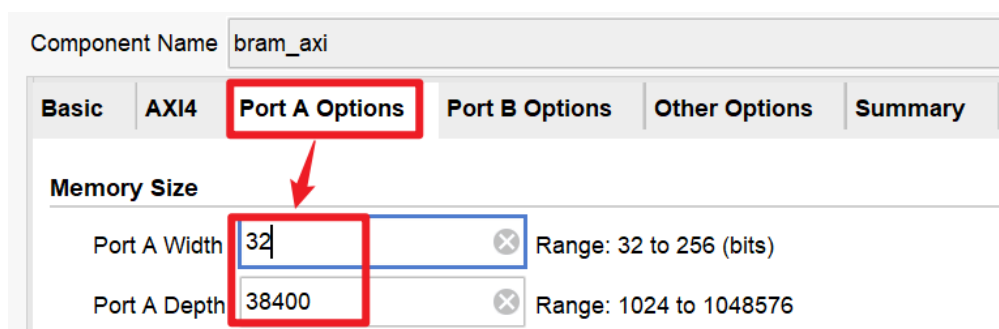


图4-3 配置主存IP核的数据位宽及容量

继续点击切换到“Other Options”标签页，勾选“Load init File”，并通过Browse按钮选择 SoC工程的 src / coe 目录下的 lw.coe 或 ld.w.coe；然后继续勾选“Fill Remaining Memory Locations”，并取消“Enable Safety Circuit”，如图4-4所示。

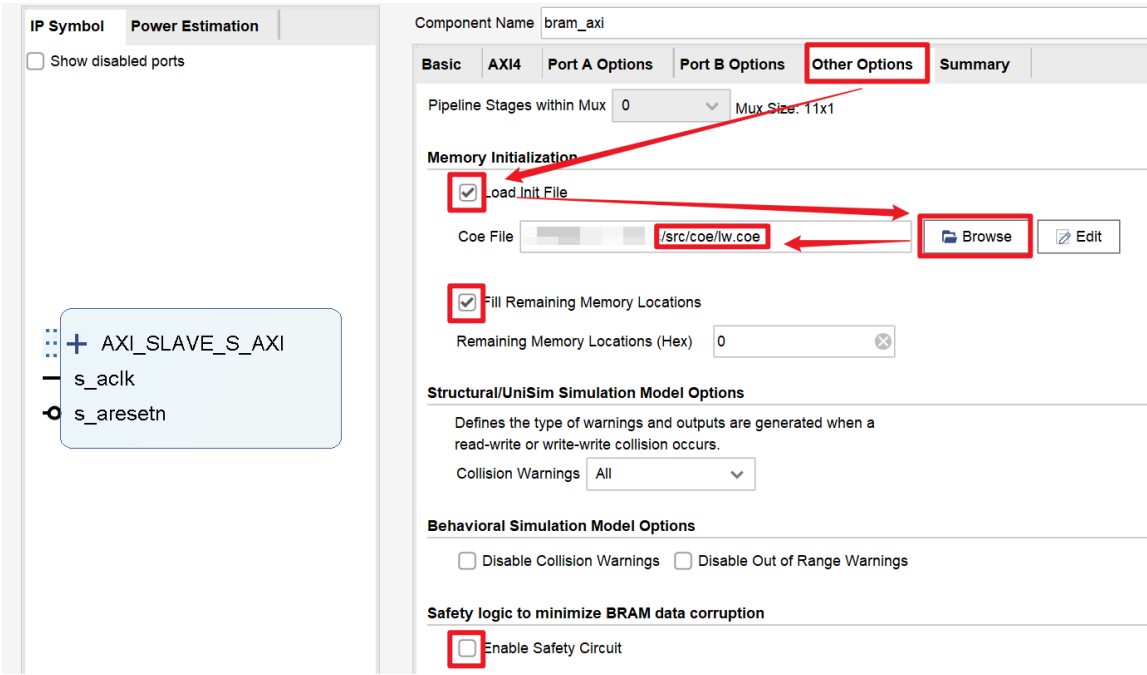


图4-4 使用.coe初始主存IP核

lw.coe 或 ld.w.coe 是使用上一节实验步骤中的 bin2coe.py 脚本对 cdp-tests / bin 中的 lw.bin 或 ld.w.bin 进行转换得到的。

接下来，在 miniRV_SoC.v 或 miniLA_SoC.v 中实例化并连接 bram_axi IP核：

```
miniRV_SoC.v 或 miniLA_SoC.v

wire [31:0] bram_awaddr ;
wire [ 7:0] bram_awlen  ;
wire [ 2:0] bram_awsz   ;
wire [ 1:0] bram_awburst;
wire      bram_awvalid;
wire      bram_awready;
wire [31:0] bram_wdata  ;
wire [ 3:0] bram_wstrb  ;
wire      bram_wlast   ;
wire      bram_wvalid  ;
wire      bram_wready  ;
wire      bram_bready  ;
wire [ 1:0] bram_bresp  ;
wire      bram_bvalid  ;
wire [31:0] bram_araddr ;
wire [ 7:0] bram_arlen  ;
wire [ 2:0] bram_arsz   ;
wire [ 1:0] bram_arburst;
wire      bram_arvalid;
wire      bram_arready;
wire      bram_rready  ;
```

4. 实现总线控制器 axi_master

基于Cache的接口时序，以及AXI的读时序和写时序，分别设计 `axi_master` 处理读请求、写请求的状态机，绘制出对应的状态转换图。

打开 `axi_master.v`，使用状态机的“三段式”结构时序AXI的读通道、写通道。

上一小节我们使用了 `lw.coe` 或 `ld.w.coe` 程序来初始化主存IP核。阅读 `cdp-tests / asm` 中的 `lw.dump` 或 `ld.w.dump` 反汇编代码可知，程序中只有读访存指令，没有写访存指令。因此，CPU执行该程序时，只会向主存发出读数据请求。

为了降低实现难度，建议同学们在实现总线控制器 `axi_master` 的时候，先实现读通道（AR、R通道）。在Vivado中通过 `lw.coe` 或 `ld.w.coe` 程序的功能仿真测试后，再实现写通道（AW、W、B通道）。

在读通道、写通道均已实现之后，可使用 `bin2cof.py` 脚本把 `sw.bin` 或 `st.w.bin` 转换成 `cof` 文件，然后将其导入 `bram_axi` 的IP核，再进行功能仿真测试。



调试建议

实现 `axi_master` 后，首次在Vivado运行功能仿真之前，建议先通过 `defines.vh` 头文件禁用Cache。等通过仿真测试之后，启用Cache并再次进行调试。

I/O接口编程

1. UART外设的I/O端口功能

根据AXI Uartlite IP核的用户手册，CPU需要通过读写表5-1所示的寄存器来控制UART串口。

表5-1 AXI Uartlite IP核的寄存器地址表

Address Offset	Register Name	Description
0h	Rx FIFO	Receive data FIFO
04h	Tx FIFO	Transmit data FIFO
08h	STAT_REG	UART Lite status register
0Ch	CTRL_REG	UART Lite control register

其中，Rx FIFO、Tx FIFO分别用于接收字符、发送字符；STAT_REG是状态寄存器，用于判断FIFO的状态、记录中断使能状态等；CTRL_REG是控制寄存器，用于初始化、中断使能等。

由实验原理2.2节的表2-1可知，UART外设的I/O接口基地址是0xFFFF_3000，并且包含4个端口，这4个端口分别与表5-1的4个寄存器一一对应。

根据用户手册，状态寄存器STAT_REG的最低4位可用于获取FIFO的状态，如表5-2所示。

表5-2 STAT_REG寄存器的比特定义

Bits	Name	Access	Reset Value	Description
3	Tx FIFO Full	Read	0h	Indicates if the transmit FIFO is full. 0 = Transmit FIFO is not full 1 = Transmit FIFO is full
2	Tx FIFO Empty	Read	01h	Indicates if the transmit FIFO is empty. 0 = Transmit FIFO is not empty 1 = Transmit FIFO is empty
1	Rx FIFO Full	Read	0h	Indicates if the receive FIFO is full. 0 = Receive FIFO is not full 1 = Receive FIFO is full
0	Rx FIFO Valid Data	Read	0h	Indicates if the receive FIFO has data. 0 = Receive FIFO is empty 1 = Receive FIFO has data

当Tx FIFO非空时，AXI Uartlite IP核会自动从Tx FIFO中取出字符并按照UART数据帧的格式进行发送。CPU需要发送字符时，必须先通过STAT_REG[3]判断Tx FIFO是否已满。若Tx FIFO已满，则CPU必须等待Tx FIFO有空闲空间时，才能把待发送的字符写入到Tx FIFO。

当AXI Uartlite IP核从RX引脚接收到字符之后，字符将被缓存到Rx FIFO。当Rx FIFO非空时，STAT_REG[0]有效，此时CPU读取Rx FIFO，就能获取到输入字符。

根据用户手册，控制寄存器CTRL_REG的最低2位可用于初始化FIFO，如表5-3所示。

表5-3 CTRL_REG寄存器的比特定义

Bits	Name	Access	Reset Value	Description
1	Rst Rx FIFO	Write	0h	Reset/clear the receive FIFO Writing a 1 to this bit position clears the receive FIFO 0 = Do nothing 1 = Clear the receive FIFO
0	Rst Tx FIFO	Write	0h	Reset/clear the transmit FIFO Writing a 1 to this bit position clears the transmit FIFO 0 = Do nothing 1 = Clear the transmit FIFO

在初始化阶段，CPU需要通过CTRL_REG[1:0]清空Rx FIFO和Tx FIFO，以免发送或接收字符时出现无效的字符数据。

2. I/O接口编程

实现miniRV SoC的同学，点击下载[c_test_rv_stu.tar.gz](#)。

实现miniLA SoC的同学，点击下载[c_test_la_stu.tar.gz](#)。

把下载的C_TEST测试包拷贝到虚拟机的 /home/usr 目录，然后在虚拟机终端执行下列命令进行解压。

```
1 tar zxvf c_test*.tar.gz
```

C_TEST测试包共包含6个C语言测试程序，如表5-4所示。其中，测试0~2、测试5均包含若干待补全的代码。待补全之处均有“TODO”注释标记。可在打开源码文件之后，使用快捷键  搜索“TODO”来定位待补全的位置。

表5-4 C_TEST测试包中的测试程序说明

序号	测试程序名称	TODO个数	说明
测试0	0_uart_test	5	通过外设读写测试UART是否工作正常
测试1	1_formatIO_test	5	测试 scanf、printf 函数是否工作正常
测试2	2_sort_test	5	测试递归、malloc 是否工作正常

序号	测试程序名称	TODO个数	说明
测试3	3_ddr_test	0	测试DDR读写是否正常
测试4	4_coremark	0	测试CPU性能
测试5	5_llama2.c	5	测试LLAMA2推理程序

参照[实验原理2.3节 I/O接口的读写访问](#)中的I/O接口访问方法，结合以上关于AXI Uartlite的端口说明，完成C_TEST测试包中的测试0~2的相关代码。

在虚拟机终端通过 `cd` 命令进入相应的测试目录，然后执行下列命令编译代码。

```
1 ./compile.sh
```

编译成功后，测试目录下将出现 `main.s`、`main.coe` 和 `main.bin` 文件，分别是程序的反汇编代码、用于初始化主存的COE文件、用于本实验下板测试的二进制程序。

3. 在FPGA上测试C程序

本实验提供已生成好比特流的SoC，用于对编写好的C程序进行下板测试。请根据指令集和所使用的开发板，在以下4个比特流中下载其中一个，如表5-5所示。

表5-5 用于测试C_TEST程序的SoC比特流

指令集	开发板	SoC比特流
miniRV	EGO1	lab2_IOfest_miniRV_ego1.bit
miniRV	Minisys	lab2_IOfest_miniRV_minisys.bit
miniLA	EGO1	lab2_IOfest_miniLA_ego1.bit
miniLA	Minisys	lab2_IOfest_miniLA_minisys.bit

打开Vivado，点击“Open Hardware Manager”，如图5-1所示。



图5-1 在Vivado首页点击“Open Hardware Manager”

连接开发板并下载提供的SoC比特流。

打开Tera Term串口工具，连接串口，如图5-2所示。



图5-2 使用Tera Term连接开发板串口

Tera Term会自动选择当前连接的串口，但如果电脑连接有多个串口，请先在设备管理器查看FPGA开发板连接的具体串口号，再按图5-2所示进行连接。

从菜单栏打开串口设置，设置波特率为115200，并关闭流控，如图5-3所示。

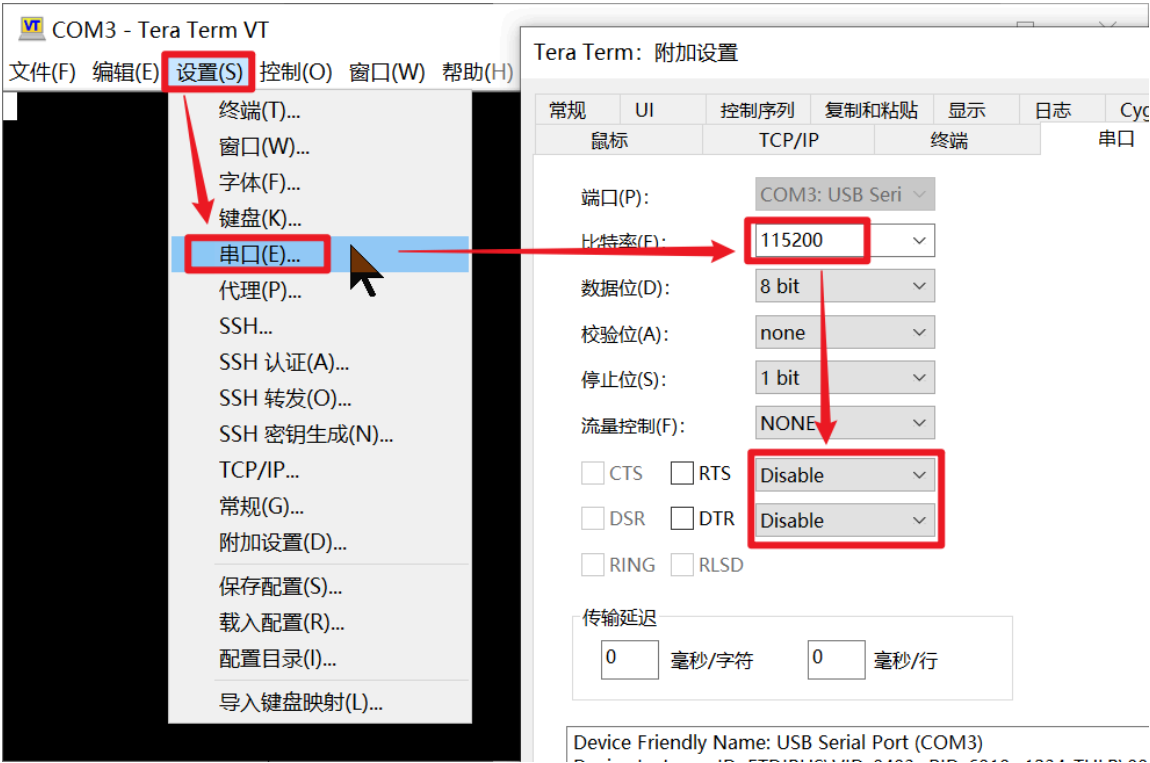


图5-3 设置UART串口的连接参数

查看[开发板使用须知](#)的图2或图3，找到复位按键。点击开发板的复位按键，随后串口终端将出现等待下载程序的提示信息，如图5-4所示。

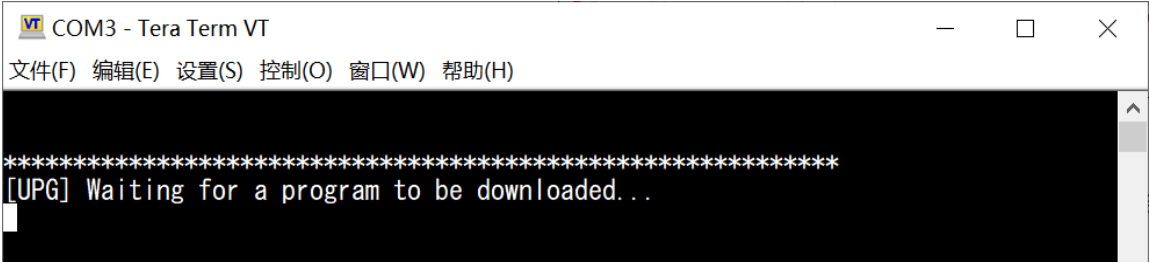


图5-4 点击开发板复位按键后，出现等待下载程序的提示信息

把编译生成的.bin文件用鼠标拖动到Tera Term界面。松开鼠标按键后，将出现发送文件的对话框。勾选以二进制格式发送，并点击确定按钮，如图5-5所示。

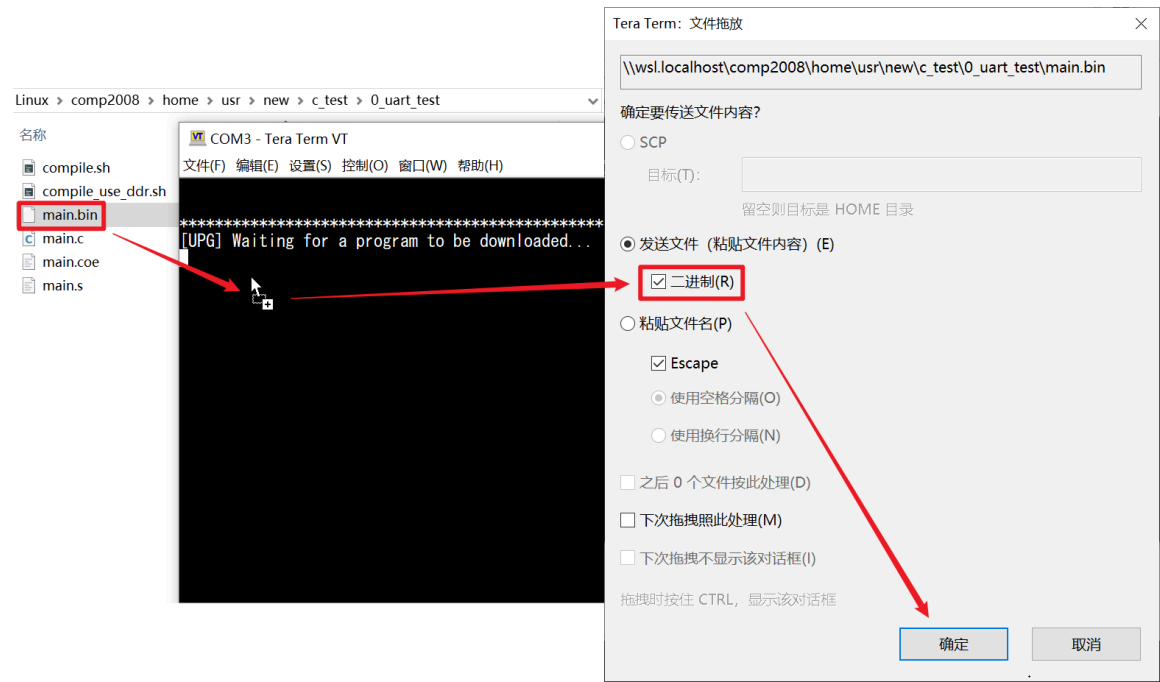


图5-5 把编译生成的程序下载到SoC运行

程序下载完毕后，将自动开始运行。根据运行结果，对所编写的C_TEST程序进行调试。

修改C程序并重新编译后，再次点击开发板的复位按键，即可继续下载并运行程序。

如果下载程序时出现异常，请重新下载比特流并重试。

I/O接口电路实现

AXI总线是常用的嵌入式总线协议。Vivado的IP核库提供了许多AXI接口的IP核，使用这些IP核可以很方便地实现外设的I/O接口电路。

1. 创建AXI协议转换器

本课程实现的外设对数据传输带宽无特别要求。Vivado为这类数据带宽不敏感的低速外设提供了支持数据读写功能的AXI GPIO IP核。我们将使用此IP核实现数码管、LED、拨码开关和计时器的I/O接口电路。

AXI GPIO IP核采用AXI4-Lite总线接口。AXI4-Lite是AXI4的精简版，信号更少、时序更简洁，但不支持猝发传输功能。

我们实现的总线控制器 `axi_master` 支持完整的AXI4总线协议。为了实现CPU通过AXI4接口访问AXI4-Lite接口的外设，我们需要使用Vivado提供的AXI协议转换器来将AXI4总线接口转换成AXI4-Lite的总线接口。

在SoC工程中，点击界面左侧的 `IP Catalog`，搜索“`protocol`”。找到 `AXI Protocol Converter` 后双击打开IP核配置窗口。保持默认设置，直接点击OK按钮即可，如图6-1所示。

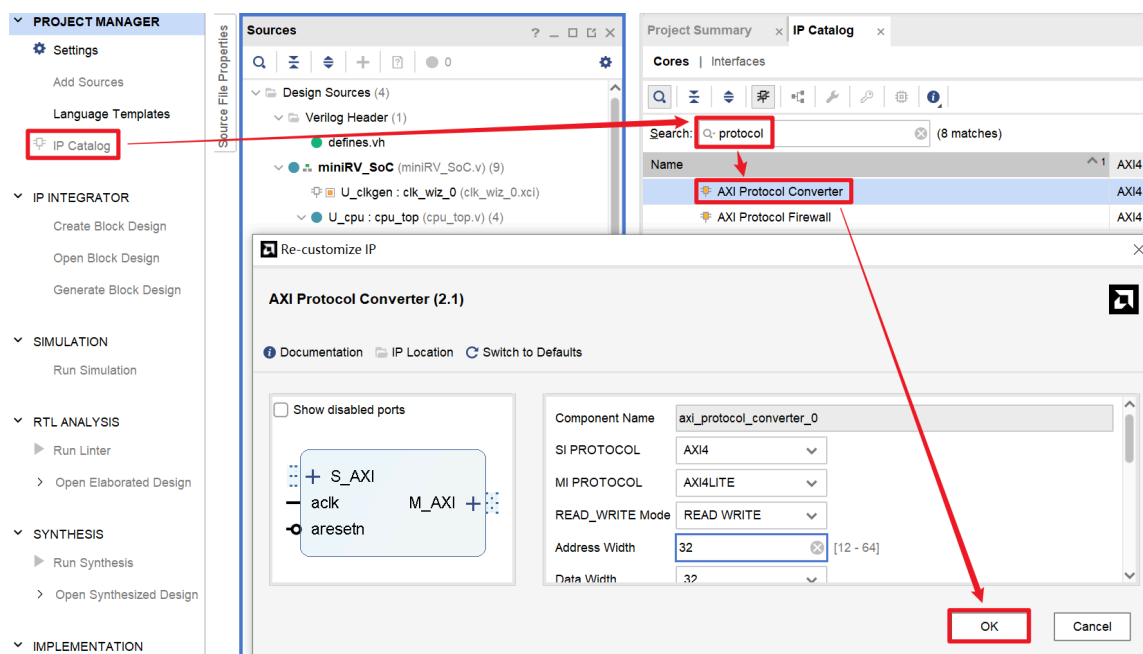


图6-1 创建AXI Protocol Converter IP核

下面以计时器和UART为例子，介绍外设I/O接口电路的设计方法。请同学们参照示例自行完成其余外设的I/O接口电路设计。

2. 计时器的I/O接口实现

在SoC工程的 `src / rtl` 目录，创建 `timer_wrap.v` 文件，并将其添加到SoC工程，如图6-2所示。

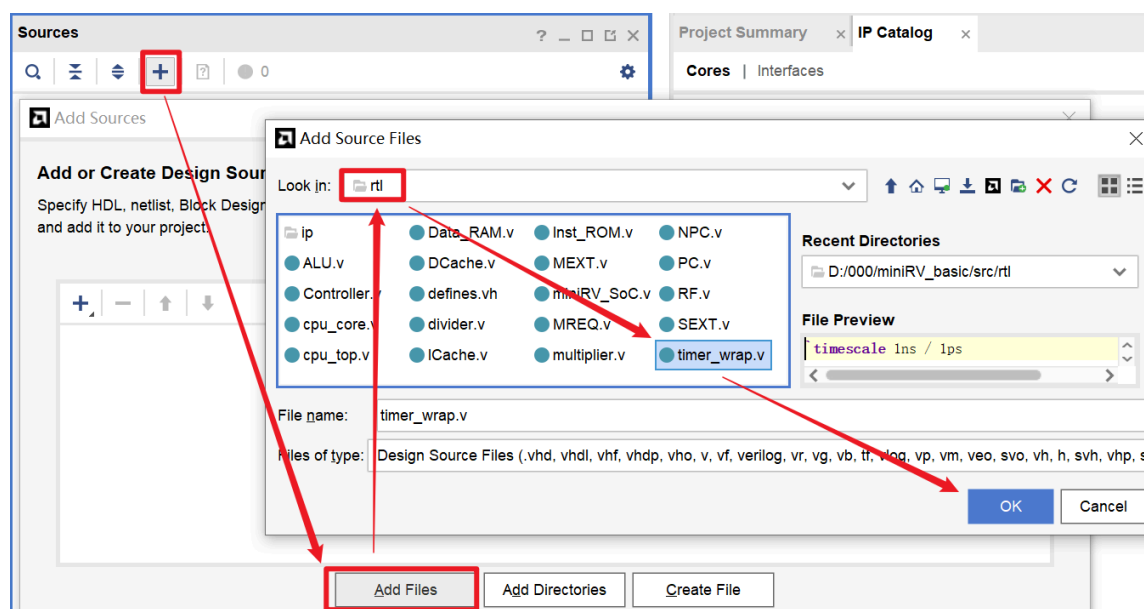


图6-2 添加计时器I/O接口模块的源文件

计时器不需要接收来自开发板上的开关输入，也不需要输出信号来驱动LED、数码管等等，因此 `timer_wrap.v` 只需要一组AXI Slave接口，如下列代码所示。

```
timer_wrap.v

module timer_wrap(
    input wire          aclk,
    input wire          aresetn,
    input wire [31:0]   s_axi_awaddr,
    input wire [ 7:0]   s_axi_awlen,
    input wire [ 2:0]   s_axi_awsz,
    input wire [ 1:0]   s_axi_awburst,
    input wire [ 0:0]   s_axi_awlock,
    input wire [ 3:0]   s_axi_awcache,
    input wire [ 2:0]   s_axi_awprot,
    input wire [ 3:0]   s_axi_awregion,
    input wire [ 3:0]   s_axi_awqos,
    input wire          s_axi_awvalid,
    output wire         s_axi_awready,
    input wire [31:0]   s_axi_wdata,
    input wire [ 3:0]   s_axi_wstrb,
    input wire          s_axi_wlast,
    input wire          s_axi_wvalid,
    output wire         s_axi_wready,
    output wire [ 1:0]  s_axi_bresp,
    output wire         s_axi_bvalid,
```

计时器的功能是给SoC提供硬件时间。在本课程中，我们使用一个64位的计数器来实现计时器，如下列代码所示。

timer_wrap.v

```
reg [63:0] timer;
always @(posedge aclk or negedge aresetn) begin
    timer <= !aresetn ? 64'h0 : timer + 64'h1;
end
```

在本课程中，计时器的I/O接口电路只需要实现把计时器的值返回给CPU即可。

miniRV和miniLA都是32位的指令集架构，SoC的总线数据宽度也是32位。因此，CPU需要两次读操作才能获取计时器的值。

接下来，我们使用AXI GPIO IP核来获取计时器的值。

点击Vivado界面左侧的 IP Catalog，搜索“gpio”。找到 AXI GPIO 后双击打开IP核配置窗口，按如图6-3所示进行配置。

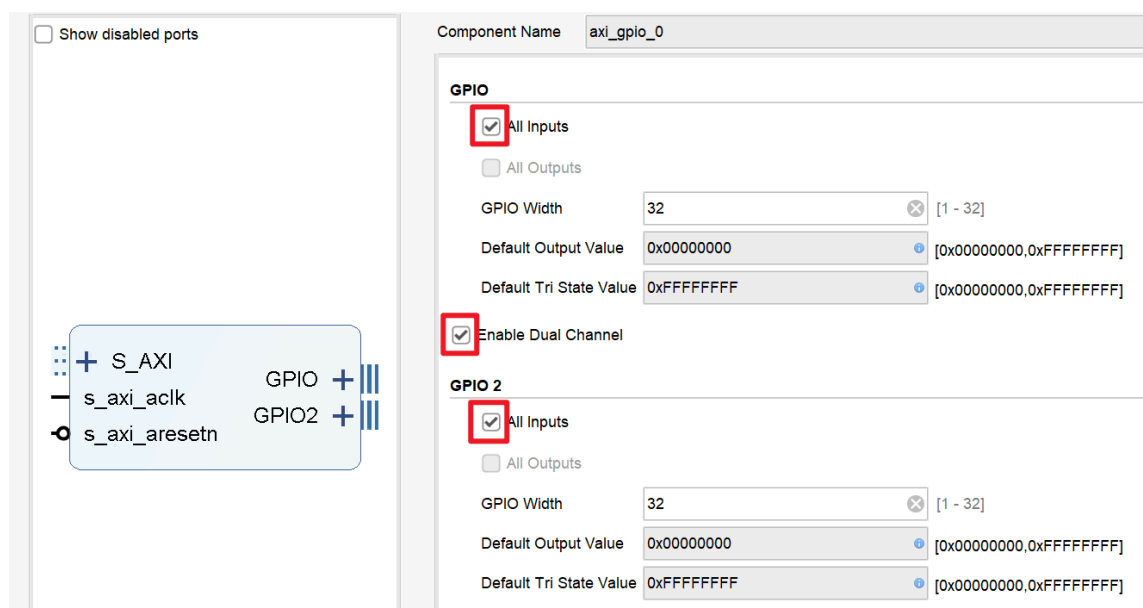


图6-3 添加AXI GPIO IP核

如果GPIO需要输出功能，则勾选“All Outputs”。对计时器而言，选择“All Inputs”即可。

在 timer_wrap.v 中实例化AXI GPIO IP核，并把 timer 寄存器的值连接到IP核的输入端口，如下列代码所示。

timer_wrap.v

```
wire [31:0] timer_awaddr;
wire        timer_awready;
wire        timer_awvalid;
wire [31:0] timer_wdata;
wire        timer_wready;
```

```

wire [ 3:0] timer_wstrb;
wire      timer_wvalid;
wire      timer_bready;
wire [ 1:0] timer_bresp;
wire      timer_bvalid;
wire [31:0] timer_araddr;
wire      timer_arready;
wire      timer_arvalid;
wire [31:0] timer_rdata;
wire      timer_rready;
wire [ 1:0] timer_rresp;
wire      timer_rvalid;

axi_gpio_0 U_timer (
    .s_axi_aclk      (aclk),
    .s_axi_aresetn   (aresetn),

```

根据AXI GPIO IP核的用户手册，GPIO端口1的偏移地址是0x0000，GPIO端口2的偏移地址是0x0008，如表6-1所示。

表6-1 AXI GPIO IP核的寄存器地址表

Address Space Offset ³	Register Name	Access Type	Default Value	Description
0x0000	GPIO_DATA	R/W	0x0	Channel 1 AXI GPIO Data Register.
0x0004	GPIO_TRI	R/W	0x0	Channel 1 AXI GPIO 3-state Control Register.
0x0008	GPIO2_DATA	R/W	0x0	Channel 2 AXI GPIO Data Register.

因此，CPU如果要读取 timer[31:0]，就需要读取计时器外设的基地址 + 0x0000的偏移地址；同理，如果要读取 timer[63:32]，就需要读取计时器外设的基地址 + 0x0008的偏移地址。

AXI GPIO IP核使得计时器的值能够通过AXI4 Lite协议来读取。在此基础上，把AXI协议转换器添加到计时器的I/O接口电路，就能实现CPU通过AXI4总线读取计时器。

在 timer_wrap.v 中实例化AXI Protocol Converter IP核，并将其与AXI GPIO IP核连接起来，如下列代码所示。

```

timer_wrap.v

axi_protocol_converter_0 U_timer_converter (
    .aclk      (aclk),
    .aresetn   (aresetn),
    .s_axi_awaddr (s_axi_awaddr),
    .s_axi_awlen  (s_axi_awlen),
    .s_axi_awsz   (s_axi_awsz),
    .s_axi_awburst (s_axi_awburst),
    .s_axi_awlock  (s_axi_awlock),
    .s_axi_awcache (s_axi_awcache),

```

```

.s_axi_awprot      (s_axi_awprot),
.s_axi_awregion    (s_axi_awregion),
.s_axi_awqos       (s_axi_awqos),
.s_axi_awvalid     (s_axi_awvalid),
.s_axi_awready     (s_axi_awready),
.s_axi_wdata       (s_axi_wdata),
.s_axi_wstrb       (s_axi_wstrb),
.s_axi_wlast       (s_axi_wlast),
.s_axi_wvalid      (s_axi_wvalid),
.s_axi_wready      (s_axi_wready),
.s_axi_bresp       (s_axi_bresp),
.s_axi_bvalid      (s_axi_bvalid),

```

最后，在 miniRV_SoC.v 或 miniLA_SoC.v 中添加 timer_wrap 模块的实例化代码。

miniRV_SoC.v 或 miniLA_SoC.v

```

wire [31:0] tim_awaddr ;
wire [ 7:0] tim_awlen  ;
wire [ 2:0] tim_awsiz  ;
wire [ 1:0] tim_awburst;
wire      tim_awvalid;
wire      tim_awready;
wire [31:0] tim_wdata  ;
wire [ 3:0] tim_wstrb  ;
wire      tim_wlast   ;
wire      tim_wvalid  ;
wire      tim_wready  ;
wire      tim_bready  ;
wire [ 1:0] tim_bresp  ;
wire      tim_bvalid  ;
wire [31:0] tim_araddr ;
wire [ 7:0] tim_arlen  ;
wire [ 2:0] tim_arsiz  ;
wire [ 1:0] tim_arburst;
wire      tim_arvalid;
wire      tim_arready;
wire      tim_rready  ;

```

3. UART的I/O接口实现

UART可以直接使用AXI Uartlite IP核来实现其I/O接口电路。

点击Vivado界面左侧的IP Catalog，搜索“uart”。找到AXI Uartlite后双击打开IP核配置窗口，按如图6-4所示进行配置。

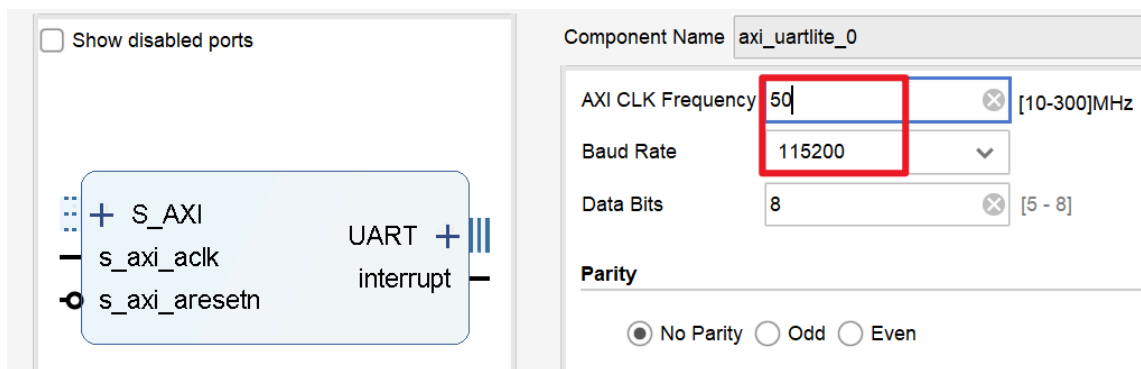


图6-4 创建AXI Uartlite IP核

其中，波特率设置为115200，而 频率则需要与CPU时钟频率一致（模板工程默认为50MHz）。若后续通过优化提高了CPU主频，必须相应地修改AXI Uartlite IP核的输入时钟频率，否则串口将不能正常工作。

创建 `uart_wrap.v`，添加AXI Uartlite IP核与AXI Protocol Converter IP核的实例化代码，然后把 `tx`、`rx` 信号添加到模块接口信号列表即可，其他与计时器相同，此处不再赘述。

4. 创建总线桥模块

到目前为止，SoC工程存在1个AXI总线的主设备（即CPU），以及6个从设备（主存和5个外设）。为了使得主设备能够同时连接多个从设备，我们需要在SoC工程中添加支持AXI协议的总线桥，即AXI Crossbar。

点击Vivado界面左侧的IP Catalog，搜索“cross”。找到AXI Crossbar后双击打开IP核配置窗口，如图6-5所示。

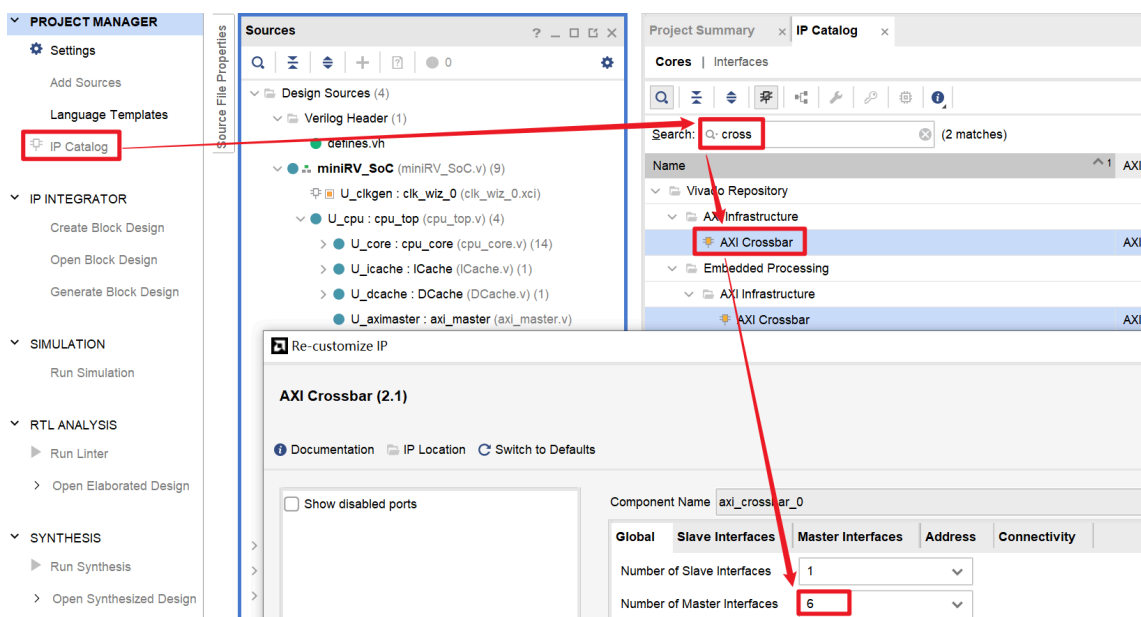


图6-5 创建AXI总线桥模块

设置Master接口的数量为6，然后点击切换到“Address”标签页，按图6-6所示设置主存和外设I/O接口的基址及地址范围。

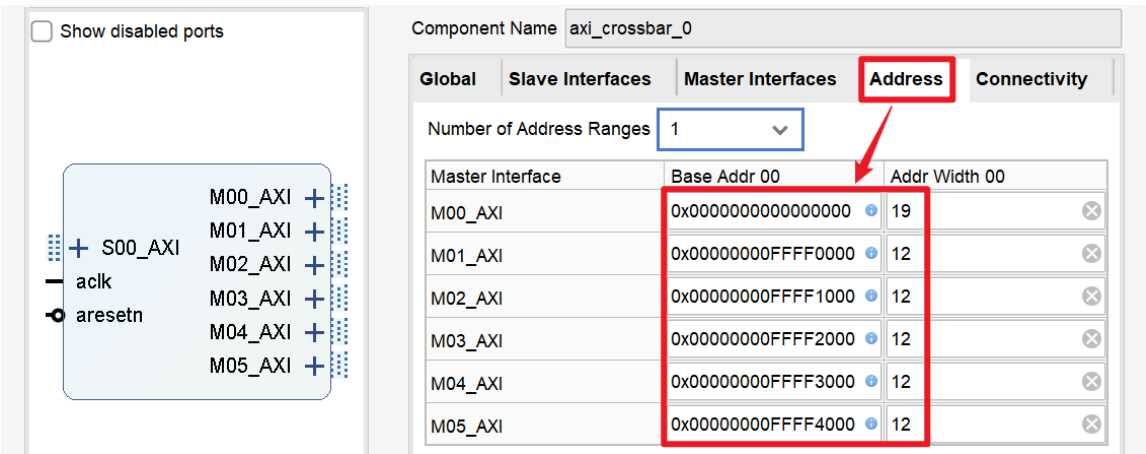


图6-6 为主存及外设I/O接口分配地址空间

接下来，在 `miniRV_SoC.v` 或 `miniLA_SoC.v` 中实例化总线桥，并把主存模块和外设I/O接口电路与总线桥连接起来，如下列代码所示。

```
miniRV_SoC.v 或 miniLA_SoC.v

`ifdef RUN_TRACE
    assign bram_awaddr  = cpu_awaddr ;
    assign bram_awlen   = cpu_awlen  ;
    assign bram_awsz    = cpu_awsz   ;
    assign bram_awburst = cpu_awburst;
    assign cpu_awready  = bram_awready;
    assign bram_awvalid = cpu_awvalid;
    assign bram_wdata   = cpu_wdata  ;
    assign bram_wstrb   = cpu_wstrb  ;
    assign bram_wlast   = cpu_wlast  ;
    assign bram_wvalid  = cpu_wvalid ;
    assign cpu_wready   = bram_wready;
    assign cpu_bresp    = bram_bresp ;
    assign cpu_bvalid   = bram_bvalid;
    assign bram_bready  = cpu_bready ;
    assign bram_araddr  = cpu_araddr ;
    assign bram_arlen   = cpu_arlen  ;
    assign bram_arsz    = cpu_arsz   ;
    assign bram_arburst = cpu_arburst;
    assign bram_arvalid = cpu_arvalid;
    assign cpu_arready  = bram_arready;
```

注意，Trace测试框架不支持IP核仿真，因此必须通过 `RUN_TRACE` 宏定义对代码进行条件编译——在运行Trace测试时，CPU的AXI接口与主存模块 `bram_axi` 直接相连，而在其他情形下，CPU通过AXI Crossbar实现6个从设备的访问。

5. 下板测试

在虚拟机中，编译上一节编写的C_TEST测试程序，把生成的.coe文件导入 `bram_axi` IP核，生成比特流并下板测试。

LLAMA2推理测试

EGO1开发板可用作主存物理介质的存储资源不足，若想在SoC上运行LLAMA2推理程序，首先需把代码移植到Minisys开发板的模板工程。

移植方法

首先，在Vivado工程设置中，把FPGA的芯片型号更换成 xc7a100tfgg484-1。

更换芯片型号之后，IP核的图标会出现红色的锁，提示我们需要更新IP核。在任意IP核上右键，点击“Report IP Status”，随后在Vivado界面下方点击“Upgrade”按钮。

下载Minisys模板工程，参照其约束文件，修改相应的引脚约束代码，去除多余的一组数码管段选信号。

参照实验1-实验原理-功能部件-3.1 Block Memory的容量修改，把 `bram_axi` IP核的容量修改为 $32\text{bit} \times 112640$ 。

参照[计算机组成原理实验2-附录B.DDR控制器的使用](#)，创建MIG IP核。在设置引脚约束时，使用[DDR3 Minisys 1.35V.ucf](#)。

把AXI Crossbar IP核的Master接口数量改为7，并按照图7-1所示修改其地址空间。

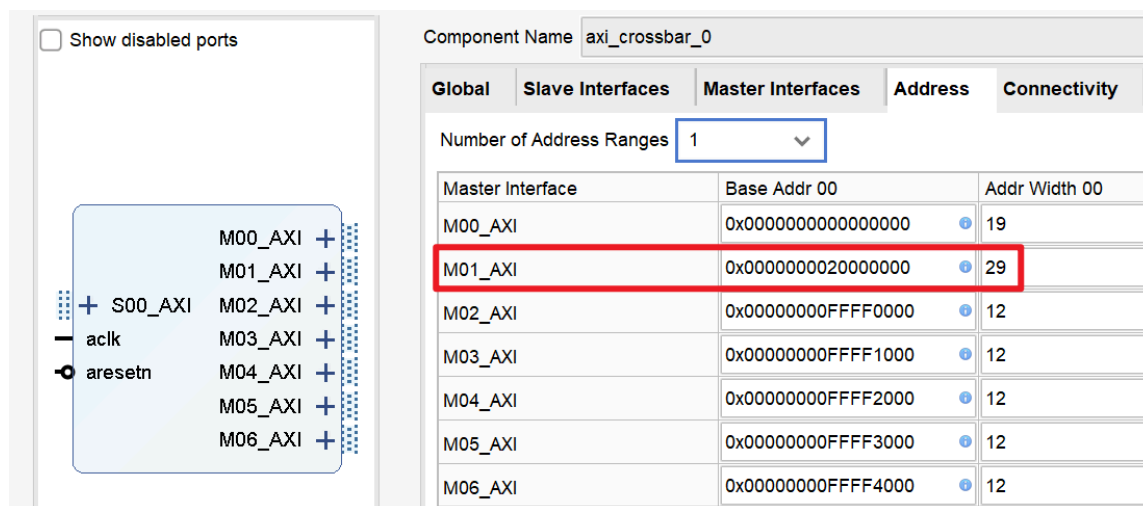


图7-1 为DDR分配地址空间

修改时钟IP核，新增用于MIG IP核的时钟信号，如图7-2所示。

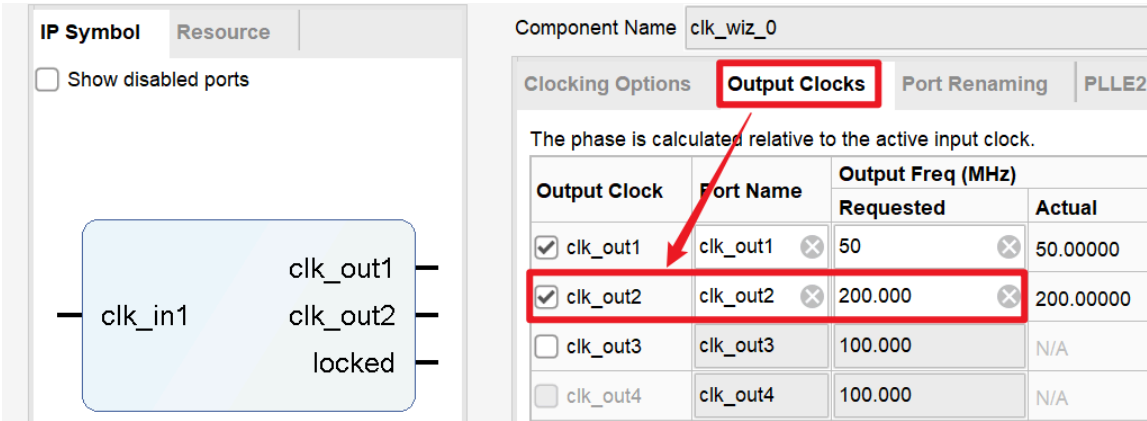


图7-2 修改时钟IP核配置，增加MIG IP核所需的时钟信号

在 miniRV_SoC.v 或 miniLA_SoC.v 中，生成MIG IP核所需的复位信号 mig_rst、时钟信号 mig_clk，并把DDR初始化完成的标志位信号 ddr_init_done 添加到SoC的复位信号逻辑当中，如下列代码所示。

```
miniRV_SoC.v 或 miniLA_SoC.v

`ifdef RUN_TRACE
    wire sys_clk = fpga_clk;
    wire sys_rst = fpga_rst;
`else
    wire pll_clk1, pll_clk2;
    wire pll_lock;
    wire sys_clk = pll_lock & pll_clk1;
    wire mig_clk = pll_lock & pll_clk2;

    wire ddr_init_done;
    wire mig_rst = fpga_rst | !pll_lock;
`ifdef USE_DDR
    wire sys_rst = fpga_rst | !pll_lock | !ddr_init_done;
`else
    wire sys_rst = fpga_rst | !pll_lock;
`endif

clk_wiz_0 U_clkgen (
    .clk_in1    (fpga_clk),
    .locked     (pll_lock),
    .clk_out1   (pll_clk1),
```

在 miniRV_SoC.v 或 miniLA_SoC.v 中实例化MIG IP核，如下列代码所示。

```
miniRV_SoC.v 或 miniLA_SoC.v

`ifndef RUN_TRACE
`ifdef USE_DDR
    mig_wrap U_mig (
        .aclk          (sys_clk),
        .aresetn        (!mig_rst),
        .s_axi_awaddr    (mig_awaddr ),
        .s_axi_awlen     (mig_awlen  ),
```

```

.s_axi_awsz (mig_awsz ),
.s_axi_awburst (mig_awburst),
.s_axi_awvalid (mig_awvalid),
.s_axi_awready (mig_awready),
.s_axi_wdata (mig_wdata ),
.s_axi_wstrb (mig_wstrb ),
.s_axi_wlast (mig_wlast ),
.s_axi_wvalid (mig_wvalid ),
.s_axi_wready (mig_wready ),
.s_axi_bresp (mig_bresp ),
.s_axi_bvalid (mig_bvalid ),
.s_axi_bready (mig_bready ),
.s_axi_araddr (mig_araddr ),

```

将MIG IP核连接到AXI Crossbar IP核，如下列代码所示。

miniRV_SoC.v 或 miniLA_SoC.v

```

axi_crossbar_0 U_bridge (
    .aclk          (sys_clk),
    .aresetn       (!sys_rst),
    .s_axi_awaddr  (cpu_awaddr ),
    .s_axi_awlen   (cpu_awlen  ),
    .s_axi_awsz    (cpu_awsz   ),
    .s_axi_awburst (cpu_awburst),
    .s_axi_awvalid (cpu_awvalid),
    .s_axi_awready (cpu_awready),
    .s_axi_awlock  (1'b0      ),
    .s_axi_awcache (4'h0      ),
    .s_axi_awprot  (3'h0      ),
    .s_axi_awqos   (4'h0      ),
    .s_axi_wdata   (cpu_wdata  ),
    .s_axi_wstrb   (cpu_wstrb  ),
    .s_axi_wlast   (cpu_wlast  ),
    .s_axi_wvalid  (cpu_wvalid ),
    .s_axi_wready  (cpu_wready ),
    .s_axi_bresp   (cpu_bresp  ),
    .s_axi_bvalid  (cpu_bvalid ),
    .s_axi_bready  (cpu_bready ),

```

为SoC顶层模块添加DDR输入输出信号，如下列代码所示。

miniRV_SoC.v 或 miniLA_SoC.v

```

`ifdef USE_DDR
    // DDR Interface
    output wire [14:0] ddr3_addr,
    output wire [ 2:0] ddr3_ba,
    output wire       ddr3_cas_n,
    output wire [ 0:0] ddr3_ck_p,
    output wire [ 0:0] ddr3_ck_n,
    output wire [ 0:0] ddr3_cke,
    output wire       ddr3_ras_n,
    output wire       ddr3_we_n,
    inout  wire [15:0] ddr3_dq,

```

```
inout wire [ 1:0] ddr3_dqs_n,  
inout wire [ 1:0] ddr3_dqs_p,  
output wire      ddr3_reset_n,  
output wire [ 0:0] ddr3_cs_n,  
output wire [ 1:0] ddr3_dm,  
output wire [ 0:0] ddr3_odt  
`endif
```

在 `clock.xdc` 中添加时钟约束语句，如下列代码所示。该语句使得Vivado在综合实现时，把MIG IP核输出的用户时钟与SoC的主时钟当作两个不同时钟域处理，从而避免形成关键路径。

clock.xdc

```
set_clock_groups -asynchronous -group [get_clocks -of_objects [get_pins  
U_clkgen/clk_out1]] \  
                                -group [get_clocks -of_objects [get_pins  
U_mig/U_mig/ui_clk]]
```

在 `defines.vh` 头文件中，增加启用DDR的宏定义语句，如下列代码所示。

defines.vh

```
3  `define ENABLE_ICACHE  
4  `define ENABLE_DCACHE  
5  `define USE_DDR
```

最后，在虚拟机中编译LLAMA2推理程序，把生成的`.coe`文件导入 `bram_axi` IP核，运行综合实现并生成比特流，再将比特流下载到任意版本的Minisys开发板进行测试。



DDR芯片的读写测试

在下板运行LLAMA2推理程序之前，可使用`ddr_test.bit`对开发板的DDR芯片进行测试。若下板后，最低三位的绿色LED依次点亮，则表示开发板的DDR芯片正常。

若测试发现开发板的DDR芯片不可用，报告任课老师，并更换开发板。

上述测试是为了确定开发板的DDR芯片是否正常，而C_TEST里面的测试3则可用于测试你的SoC能否正确读写DDR的数据。

实现DDR控制器之后，可使用每个C_TEST测试目录下的 `compile_use_ddr.sh` 脚本编译程序，将C_TEST测试0~2重新下板运行，以确保SoC能够正确访问DDR。

LLAMA2运行效果示例

SoC下板运行LLAMA2的效果如图7-3所示。

```
Input prompt: abc
abcidents is round and fault in the sky. It was a delicious colorful face. This colors
flew to the floor.
achieved tok/s: 0.421245
```

① 单词正确，句子能构成一定含义，表示功能正确
② 性能数据

图7-3 LLAMA2下板运行效果示意图

在计算机组成原理实验中，我们曾对LLAMA2推理程序进行下板测试。当时提供给同学们的SoC工程使用静态RAM作为主存，其数据访问延迟远低于DDR，因此推理性能也明显高于使用DDR作为主存的SoC。

注意：提升CPU时钟频率后，需相应修改C程序的频率参数

提高CPU时钟频率的方法见[常见问题 - 1.7 如何提高CPU主频？](#)

模板工程的CPU时钟默认为50MHz。改变频率后，需要相应地修改C程序中的频率参数，否则程序计时不准确。

具体地，我们需要修改LLAMA2源文件 `peripheral.h` 第6行的 `CPU_CLK_FREQ` 参数，使其等于实际的CPU频率：

```
c_test / 5_llama2.c / peripheral.h

6 #define CPU_CLK_FREQ 50 // MHz
7 #define CLKS_PER_SEC (1000000 * CPU_CLK_FREQ)
```